Cardiff Metropolitan University
B.Sc. (Hons) in Software Engineering
Assignment Cover Sheet

Student Details (Student should fill the content)				
Name	Hollu Pathiranage Aruna Nuwan Srinath Kaldera			
Student ID	CL/BSCSD/34/94			
Scheduled unit details				
Unit code	CIS6003			
Unit title	Advanced Programming			
Unit enrolment details	Year	3		
	Study period	1 Year		
Lecturer				
Mode of delivery	Full Time			
Assignment Details				
Nature of the Assessment	WRIT1			
Topic of the Case Study	Sunrise Dental Clinic			
Learning Outcomes covered				
Word count	9321			
Due date / Time	05th September 2026			
Extension granted?	~~Yes~~	No	Extension Date	
Is this a resubmission?	~~Yes~~	No	Resubmission Date	
Declaration				
I certify that the attached material is my original work. No other person's work or ideas have been used without acknowledgement. Except where I have clearly stated that I have used some of this material elsewhere, I have not presented it for examination / assessment in any other course or unit at this or any other institution				
Name/Signature	Aruna Kaldera		Date	05th September 2026
Submission				
Return to:				
Result				
Marks by 1st Assessor		Name & Signature of the 1st Assessor		**Agreed Mark**
Marks by2nd Assessor		Name & Signature of the 2nd Assessor		
Comments on the Agreed mark				

st20286243 CIS6003 WRIT1.pdf

by Hollu Pathiranage Aruna Nuwan Srinath Kaldera

Submission date: 05-Sep-2026 09:55AM (UTC+0100)
Submission ID: 288329464
File name: 128773_Hollu_Pathiranage_Aruna_Nuwan_Srinath_Kaldera_st20286243_CIS6003_WRIT1_3150916_412017576.pdf (2.25M)
Word count: 9321
Character count: 54053

TASK A – SYSTEM DESIGN WITH UML DIAGRAMS

INTRODUCTION

Sunrise Dental Clinic currently manages patient appointments and treatment records on paper. This causes double bookings, lost records, long waiting times, and billing mistakes. I designed the system on paper first, before writing any code, using three UML diagrams that look at the system from three angles: a Use Case diagram for what each user can do, a Class diagram for the data and behaviour the system needs, and a set of Sequence diagrams for how the different parts of the system talk to each other. I kept these diagrams updated as the build went on, so they match what was actually implemented. Where I made an assumption the brief did not state, I explain it below.

1. USE CASE DIAGRAM

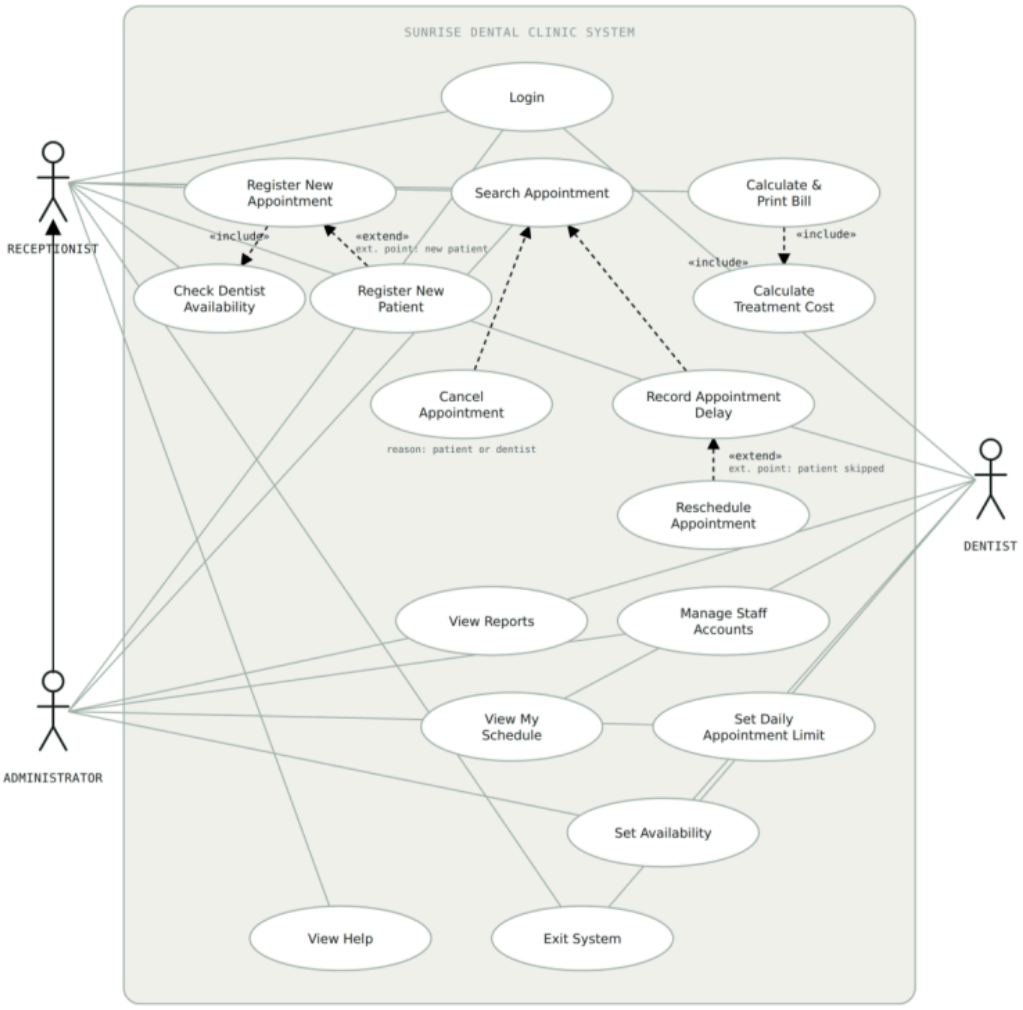

Figure 1: Use Case Diagram for the Sunrise Dental Clinic System.

Actors

- **Receptionist** – the main staff user, handles day-to-day bookings, search, and billing.

- **Administrator** – I modelled this as a generalisation of Receptionist, because an administrator can do everything a receptionist can, plus manage staff accounts, view reports, and override a dentist's settings.
- **Dentist** – not in the original brief, but the brief allows extra functionality if it is explained. I added this actor because a real clinic needs the dentist to see their schedule, report their own delays, and set their own capacity and availability. This role also let me show role-based access control in Task B.

Use cases and relationships

The system has seventeen use cases. The first six map onto the six functions in the brief (Login, Register New Appointment, Display Appointment Details, Calculate & Print Bill, Help, Exit System); I split some into smaller use cases where it made the design clearer.

I used `<<include>>` where a use case always needs another to run, and `<<extend>>` where it only sometimes branches off:

- **Register New Appointment `<<include>>` Check Dentist Availability** – always runs, to stop double booking.
- **Register New Patient `<<extend>>` Register New Appointment** – only when the patient is new.
- **Calculate & Print Bill `<<include>>` Calculate Treatment Cost** – the bill always needs a cost first.
- **Cancel Appointment** and **Record Appointment Delay** both `<<include>>` **Search Appointment**, since both need to find the appointment first.
- **Reschedule Appointment `<<extend>>` Record Appointment Delay** – only when a patient's delay is recorded as "Skip" (explained below).
- **Administrator generalises Receptionist.**

Assumptions I made

Delays, without a waiting-room subsystem. The brief does not say how to handle delays. I decided not to build a full waiting-room/queue system, since that adds complexity the brief does not ask for. I assumed patients physically wait as normal, and the system just records delays and shows staff the current schedule. A **dentist-caused delay** cascades automatically to that dentist's remaining appointments that day, since there is no real choice to make. A **patient-caused delay** needs a manual **Wait/Skip** decision from the receptionist: Wait behaves like a dentist delay from that point; Skip hands the appointment to Reschedule Appointment instead of losing it. Every appointment also has a default duration based on its treatment type, which feeds into the availability check.

Dentist capacity and availability. Also not in the brief, but a normal part of running a clinic. A dentist (or an administrator, as an override) can set a daily limit and mark unavailable periods. I did not draw an `<<include>>/<<extend>>` link between these and Check Dentist

Availability, since a use case diagram shows who triggers an action, not how earlier data is used later — that connection is shown in the class and sequence diagrams instead.

2. CLASS DIAGRAM

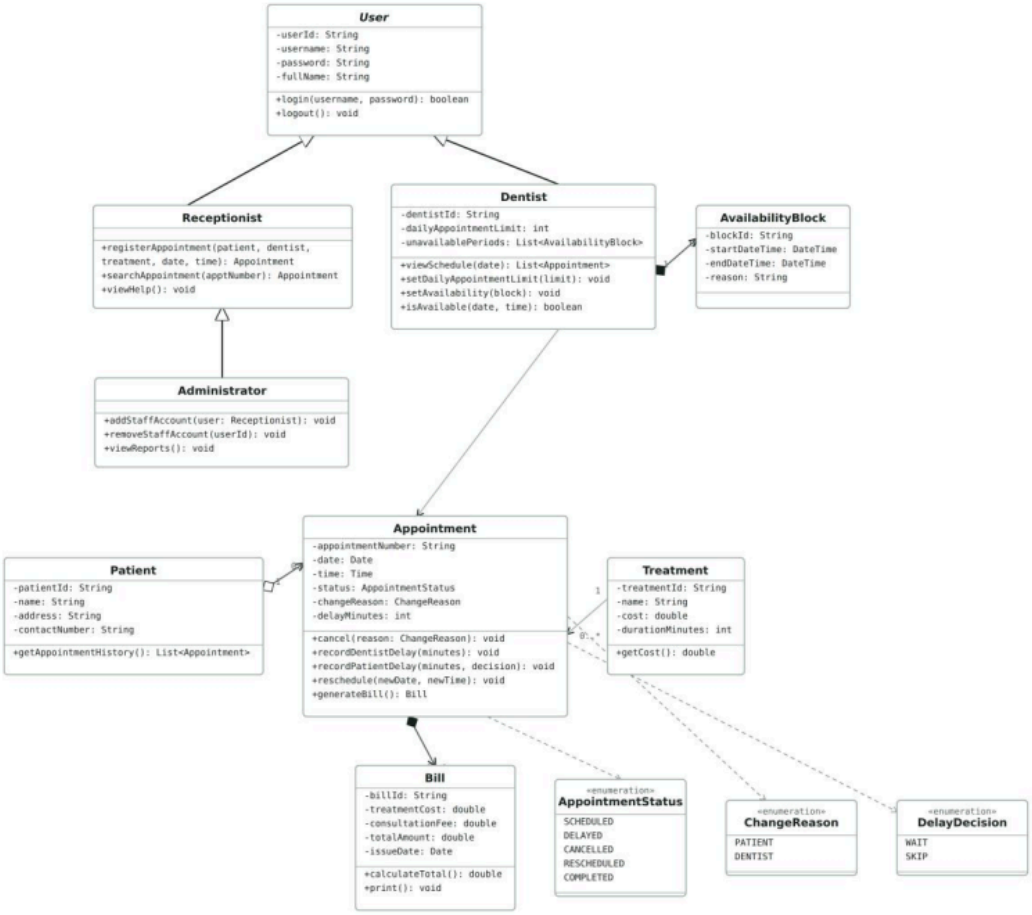

Figure 2: Class Diagram for the Sunrise Dental Clinic System.

13

The class diagram shows the static structure: what objects exist, what data they hold, and how they connect. I used standard UML visibility notation throughout — attributes are - (private) and methods are + (public), so every class only exposes behavior through its methods rather than letting other classes change its data directly.

Classes

User (abstract) holds the shared login fields and `login()`/`logout()` methods, since all staff authenticate the same way. **Receptionist** extends User with booking/search/help methods. **Administrator** extends Receptionist, mirroring the actor generalization, and adds staff-account and reporting methods. **Dentist** extends User with its own capacity/availability fields and methods. **Patient** holds contact details and appointment history. **Appointment** is the central class — it holds the status, change reason, and delay minutes, and every state-changing method (`cancel()`, the two delay methods, `reschedule()`, `generateBill()`). **Treatment**

holds cost and duration. **Bill** holds the cost breakdown and total. **AvailabilityBlock** holds a blocked date/time range. Three enumerations (`AppointmentStatus`, `ChangeReason`, `DelayDecision`) cover the fixed states each of those fields can take.

Relationships, multiplicity, and navigability

I picked a different relationship type for each pair, based on whether one object could sensibly exist without the other, rather than defaulting everything to a plain association:

- **Patient–Appointment**: aggregation (`1` to `0..*`) — each can exist without the other.
- **Appointment–Bill** and **Dentist–AvailabilityBlock**: composition — a bill has no meaning without its appointment, and a block has no meaning without its dentist, so both should be removed with their owner.
- **Dentist–Appointment** and **Treatment–Appointment**: plain association — dentists and treatments are independent records many appointments refer to.
- **Appointment → AppointmentStatus/ChangeReason/DelayDecision**: dependency, since Appointment uses these as attribute types rather than owning separate objects.

Every relationship is navigable in one direction only, and all of them point toward Appointment, since it is the record every other class needs to look up — this matches how the Task B database is actually queried.

Assumptions I made

Every state-changing action lives on Appointment itself, not on Receptionist or Dentist, so Appointment stays responsible for its own valid state changes. The two delay methods stay separate rather than merging into one, since a dentist delay always cascades automatically while a patient delay needs a Wait/Skip decision. I also assumed the consultation fee is a single flat amount for the whole clinic rather than something that changes per dentist or treatment, since this matches how many small clinics bill and needs no new use case. This fee is not shown as its own class, since no use case gives a user direct control over it — it behaves more like a clinic setting than a domain entity.

3. SEQUENCE DIAGRAMS

I drew nine sequence diagrams, one per main use case. Every diagram follows the same three-tier shape: a Servlet handles the request, a Service class carries out the business logic, and a DAO talks to the database. I used this shape from the start, before any code existed, since it is the layered architecture Task B requires (Java Servlets, Tomcat, a proper database) — so the diagrams and the working system tell the same story.

3.1 — Login.

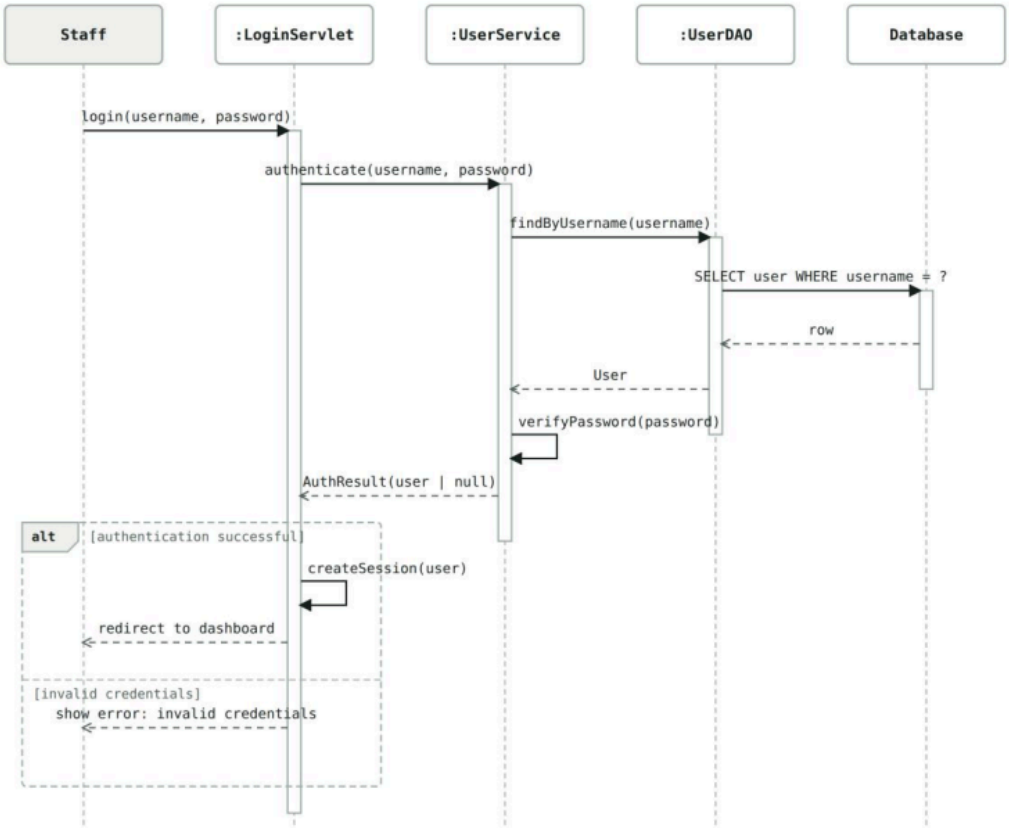

Figure 3: Sequence Diagram — Login.

The servlet asks the Service layer to verify the password through the DAO. On success it creates a session and redirects; on failure it returns an error.

3.2 — Register New Appointment.

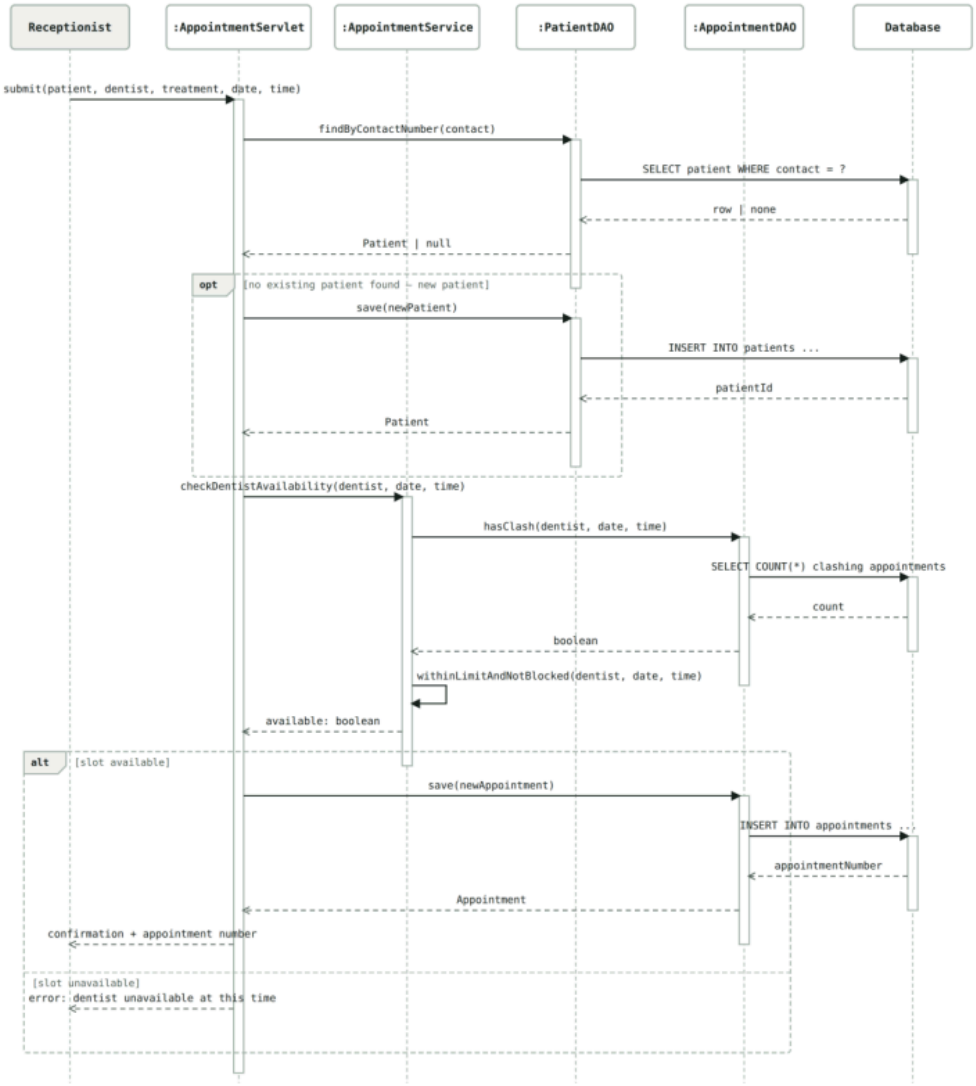

Figure 4: Sequence Diagram — Register New Appointment.

The servlet finds the patient or registers a new one, then the Service layer checks three things together: no clash with an existing appointment, the slot is not inside a blocked period, and the dentist is under their daily limit. If all pass, the appointment is saved; otherwise the system returns an "unavailable" message.

3.3 — Record Appointment Delay.

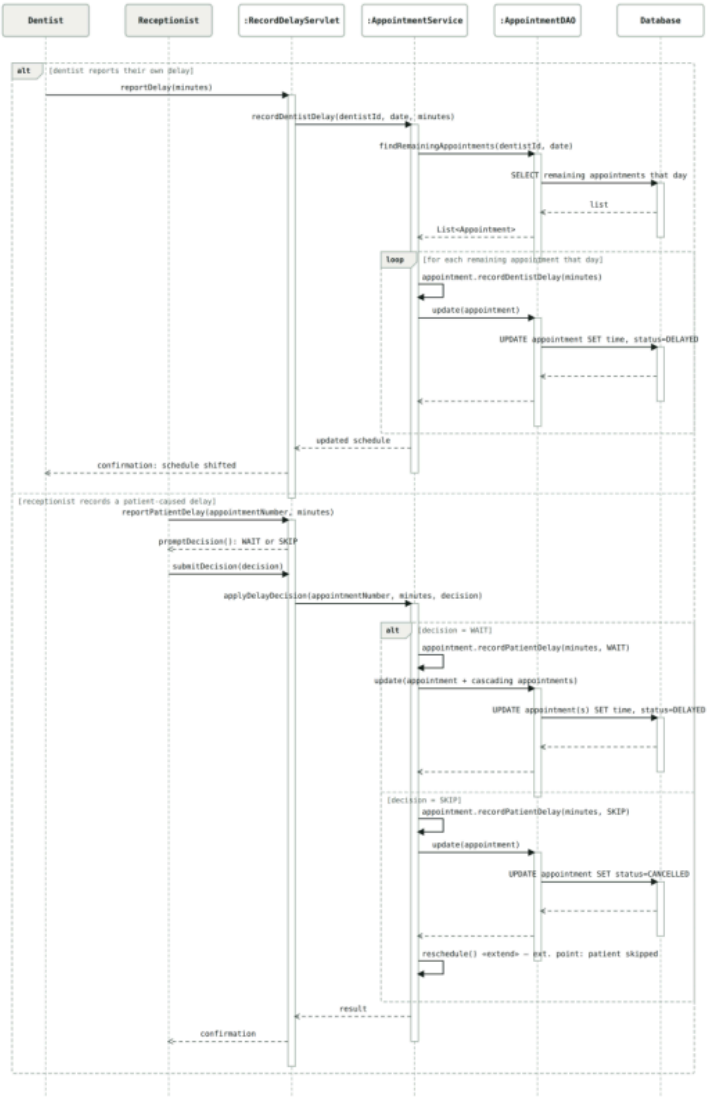

Figure 5: Sequence Diagram — Record Appointment Delay.

Two branches: a dentist's own delay cascades automatically to their remaining appointments that day; a patient's delay asks for a Wait/Skip decision, where Wait cascades the same way and Skip hands off to rescheduling.

3.4 — Cancel Appointment.

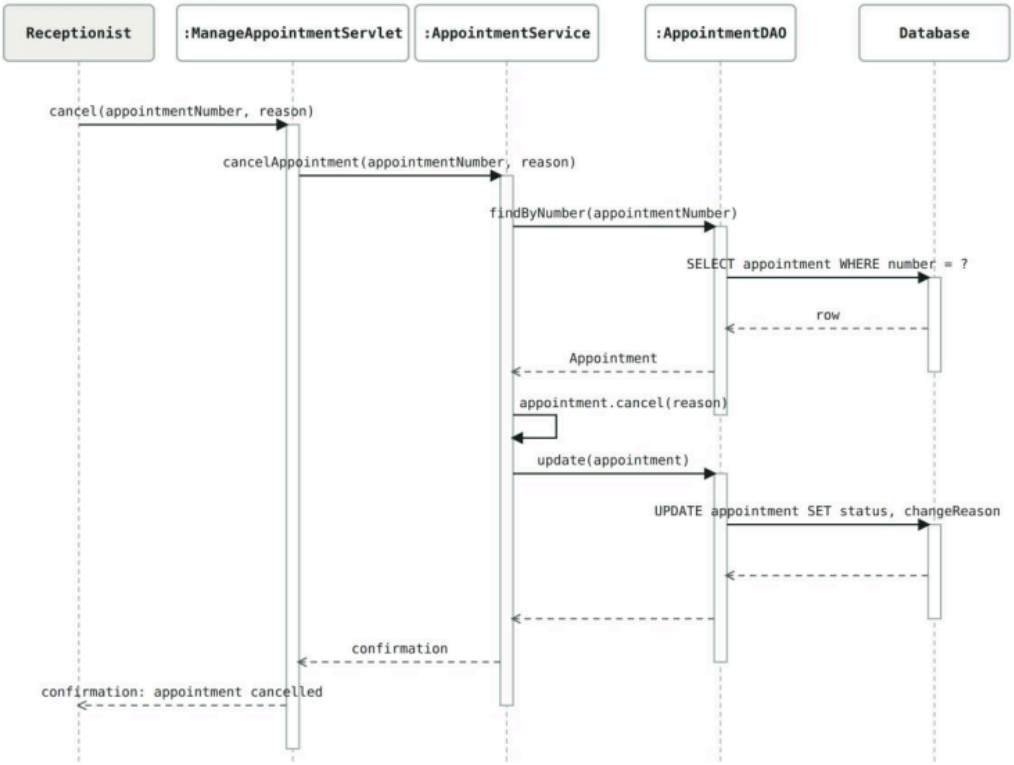

Figure 6: Sequence Diagram — Cancel Appointment.

The simplest flow — the appointment status is set to Cancelled with a reason, and the DAO saves the change.

3.5 — Reschedule Appointment.

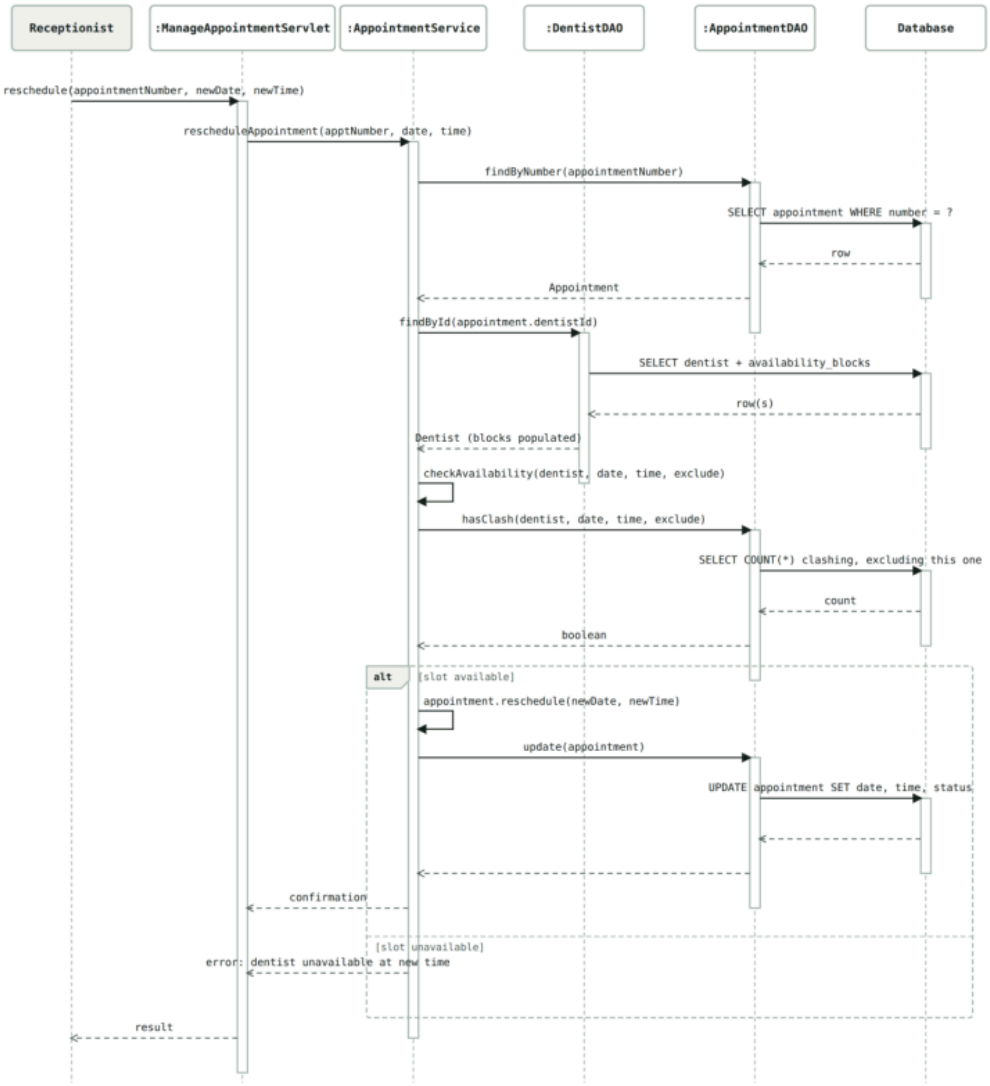

Figure 7: Sequence Diagram — Reschedule Appointment.

Re-runs the same availability check as registration, but ignores the appointment being moved so it does not clash with its own old slot. Ends in either the new time being saved, or an error if it is unavailable.

3.6 — Calculate & Print Bill.

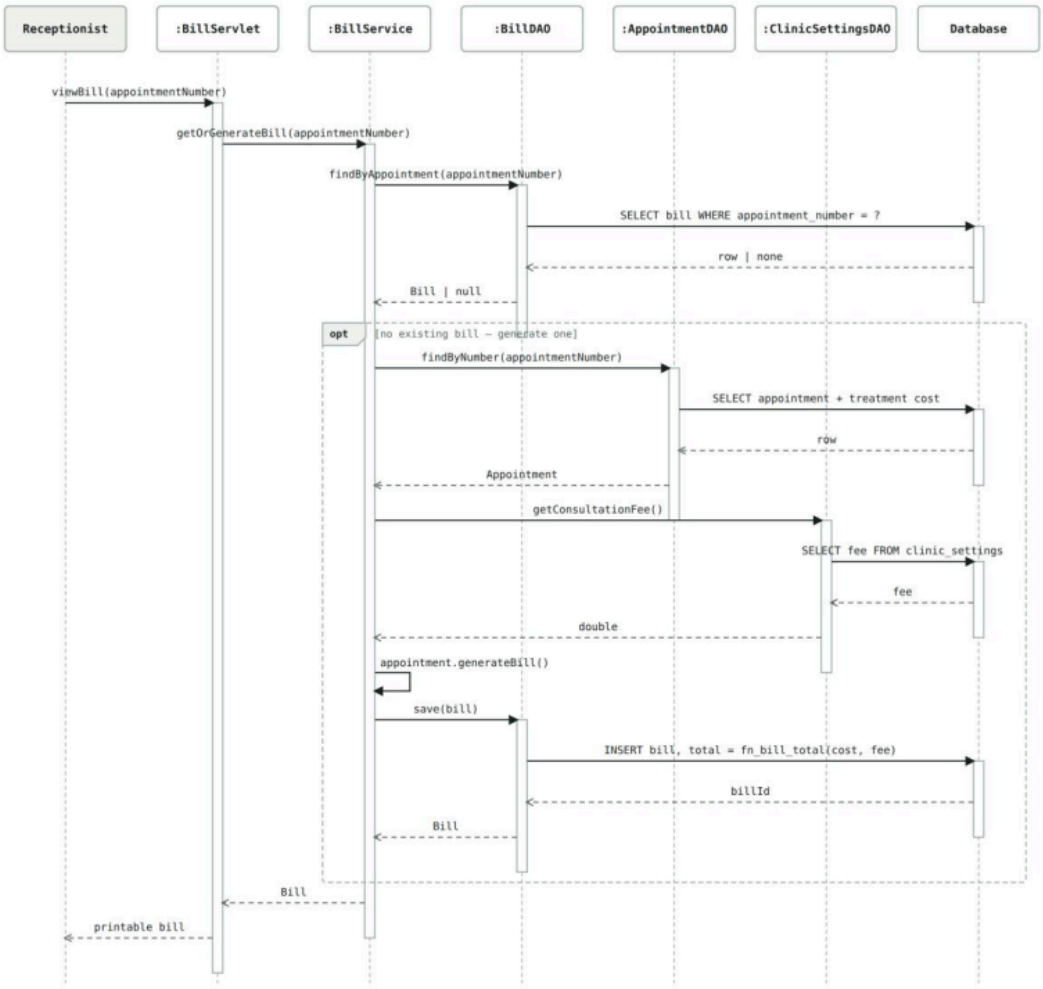

Figure 8: Sequence Diagram — Calculate & Print Bill.

If no bill exists yet, the Service layer looks up the appointment and the clinic's flat consultation fee, then the total is calculated inside the database using a function, so the stored figure can never drift from the formula.

3.7 — Manage Staff Accounts.

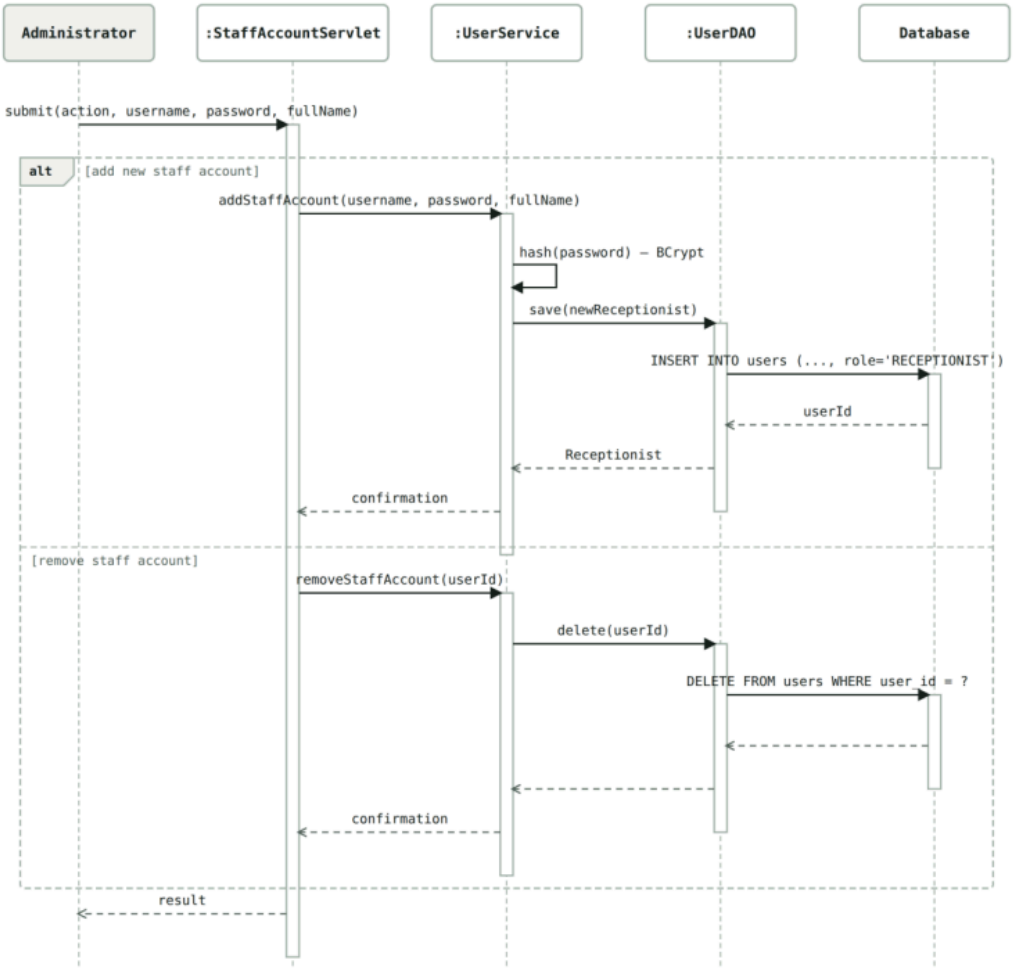

Figure 9: Sequence Diagram — Manage Staff Accounts.

An administrator adds or removes a staff account. Passwords are hashed before being saved. Both actions are Administrator-only.

3.8 — View Reports.

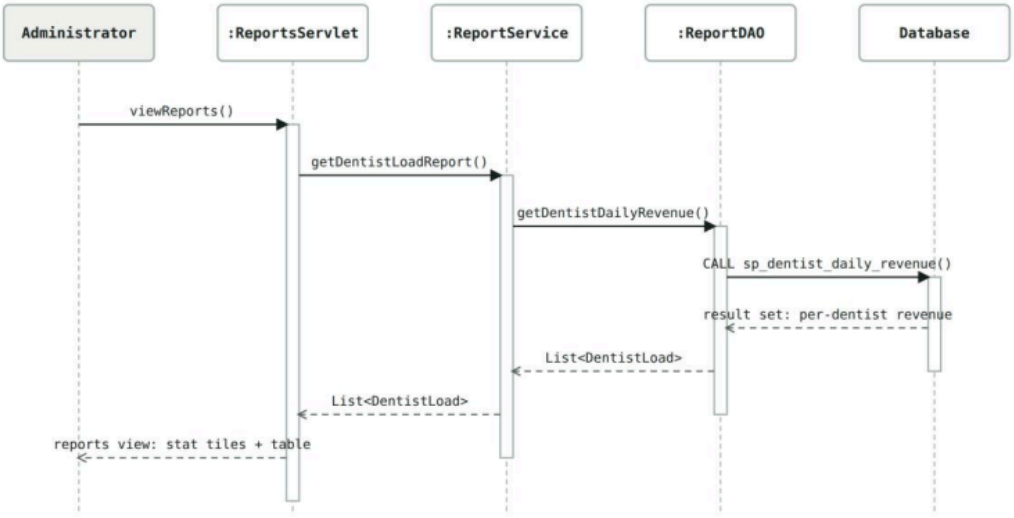

Figure 10: Sequence Diagram — View Reports.

The administrator requests the dentist load report, and a stored procedure in the database works out each dentist's daily revenue.

3.9 — Set Daily Appointment Limit / Set Availability.

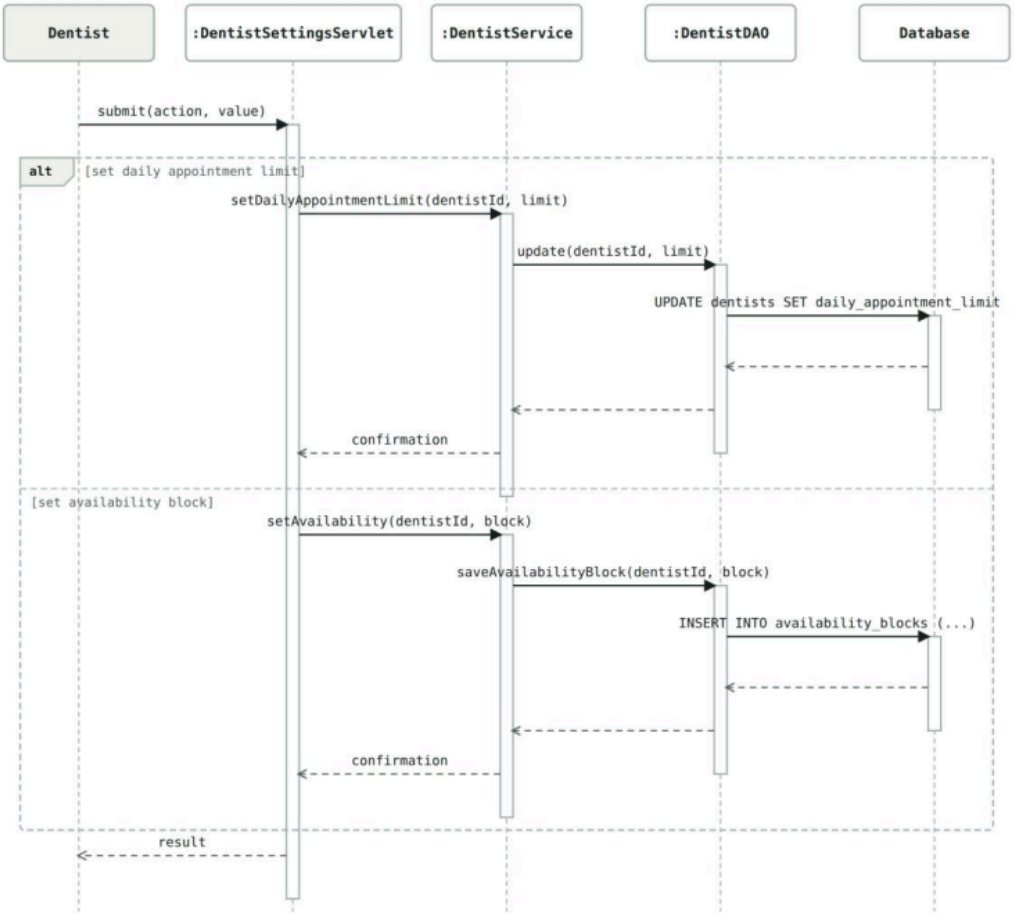

Figure 11: Sequence Diagram — Set Daily Appointment Limit / Set Availability.

A dentist (or an administrator, as an override) either changes their daily limit or adds an availability block; both go through the Service layer to the database.

4. HOW THE THREE DIAGRAM TYPES SUPPORT EACH OTHER

Each diagram answers a different question about the same system. The Use Case diagram answers "who can do what" and defines how actions depend on each other through `<<include>>/<<extend>>`. The Class diagram answers "what does the system need to remember" and turns each use case's data needs into real classes and relationships, chosen to match how the objects actually depend on each other. The Sequence diagrams answer "how does a request actually flow", showing the use case diagram's relationships and the class diagram's classes working together through the servlet/service/DAO layers.

The strongest part of this design, I think, is that the three diagrams stay consistent with each other — every `<<include>>/<<extend>>` from the use case diagram shows up again in the matching sequence diagram, and every class does the job its methods say it should. One limitation I am aware of: a few sequence diagrams simplify some method calls for readability,

leaving out a parameter the real implementation does use, and next time I would note this directly on the diagram rather than only in supporting notes. I also left a few supporting pieces — the clinic's consultation fee and the notification records — off the class diagram on purpose, since no use case gives a user direct control over them. I think this keeps the class diagram focused on the real domain model, but it does mean the class diagram is not a complete map of every table the finished database ends up with.

TASK B – IMPLEMENTATION: ARCHITECTURE, DESIGN PATTERNS AND DATABASE

INTRODUCTION

This part covers how I actually built the system to match the design in Task A, and why I made the technical choices I made. The brief asks for a distributed application with web services, proper design patterns, and a proper database, so I have organised this section around those three things, plus the validation, reporting, and session/cookie handling the marking rubric also asks about. Where something is a genuine limitation rather than a finished feature, I say so directly instead of overselling it — I would rather be marked on an honest account of what I built than have a marker find a gap I did not mention.

1. ARCHITECTURE — A DISTRIBUTED, THREE-TIER WEB APPLICATION

The system runs as three separate processes talking to each other over a network connection, not one monolithic program: a web browser (the client), Apache Tomcat running my Java code (the application server), and a MySQL server (the database server). This is what makes it a distributed, client-server application rather than a single desktop program that reads and writes a database file directly. I did not build a separate REST or SOAP API layer on top of this, because every one of the brief's six core functions is a simple staff-facing screen (log in, book an appointment, look one up, print a bill, get help, log out) — there is no second client (a mobile app, another company's system) that would actually need a JSON or XML API to talk to. Adding one would have meant writing code with no real caller, just to tick a box. The "web services" part of the requirement is met honestly: the browser talks to the server only over HTTP, through named URL routes, and the server never talks to the database except through its own data-access layer.

Inside the application server, I kept the same three layers all the way through, matching the sequence diagrams from Task A:

- **Servlet (presentation tier)** — reads the HTTP request, checks the user is allowed to be there, and picks what to show next. Eleven `@WebServlet` classes cover the whole application (see Table 1). Every one of them forwards to a JSP page rather than building HTML by hand.
- **Service (business logic tier)** — the actual rules live here: is this slot free, has this dentist hit their daily limit, is this delay the dentist's fault or the patient's. Servlets never contain business rules themselves.
- **DAO (data access tier)** — every entity has its own DAO, so the Service tier never writes SQL directly. This is also design pattern 1 below.

```
src/main/java/com/sunrisedental/
├── web/           (presentation tier — 11 servlets + RoleGuard,
AuthenticationFilter)
|   ├── LoginServlet.java
|   ├── LogoutServlet.java
|   ├── DashboardServlet.java
|   ├── AppointmentServlet.java
|   ├── SearchAppointmentServlet.java
|   ├── BillServlet.java
|   ├── ManageAppointmentServlet.java
|   ├── StaffAccountServlet.java
|   ├── DentistSettingsServlet.java
|   ├── ReportsServlet.java
|   ├── HelpServlet.java
|   ├── RoleGuard.java
|   └── AuthenticationFilter.java
├── service/       (business logic tier)
|   ├── UserService.java
|   ├── AppointmentService.java
|   ├── BillService.java
|   ├── DentistService.java
|   ├── ReportService.java
|   ├── NotificationService.java
|   ├── NotificationChannel.java       (Strategy interface)
|   └── SmsNotificationChannel.java    (Strategy implementation)
├── dao/           (data access tier — one interface per entity)
|   ├── UserDAO.java, PatientDAO.java, TreatmentDAO.java, DentistDAO.java,
|   |   AppointmentDAO.java, BillDAO.java, ClinicSettingsDAO.java,
|   |   NotificationDAO.java, ReportDAO.java, DentistLoad.java
|   └── impl/      (one JDBC implementation per interface above)
├── db/
|   └── DBConnectionManager.java       (Singleton)
└── model/         (the class-diagram entities — User, Receptionist,
                     Administrator, Dentist, Patient, Appointment,
Treatment,
                     Bill, AvailabilityBlock, Notification, and 3 enums)
```

Figure 12: Package structure showing the three-tier layering (presentation, business logic, data access).

Table 1: The eleven servlets and the routes they serve.

Servlet	Route	Purpose
LoginServlet	`/login`	Authenticate and start a session
LogoutServlet	`/logout`	End a session
DashboardServlet	`/dashboard`	Role-scoped home screen
AppointmentServlet	`/appointments/new`	Register a new appointment
SearchAppointmentServlet	`/appointments/search`	Display appointment details

BillServlet	`/appointments/bill`	Calculate and print a bill
ManageAppointmentServlet	`/appointments/manage`	Cancel, delay, reschedule
StaffAccountServlet	`/staff`	Add/remove staff accounts
DentistSettingsServlet	`/dentist/settings`	Daily limit and availability
ReportsServlet	`/reports`	Aggregated clinic reports
HelpServlet	`/help`	Help section

One honest limitation on the presentation side: the UI is plain HTML and CSS (a single hand-written stylesheet), not a JavaScript framework or a component library. I decided this was the right trade-off for the time I had — the brief marks working, validated functionality and a proper architecture, not a polished commercial front end, and every screen still renders correctly and consistently across the whole application.

2. DESIGN PATTERNS

I looked for places in the design where a recognised pattern genuinely fit a real problem, rather than adding a pattern just to have more patterns to write about. Three ended up in the finished code.

2.1 Data Access Object (DAO)

Every entity (`User`, `Patient`, `Treatment`, `Dentist`, `Appointment`, `Bill`, `ClinicSettings`, `Notification`) has a DAO interface plus one JDBC implementation, e.g.:

```
public interface UserDAO {
    tional<User> findByUsername(String username);
    Optional<User> findById(int userId);
    List<User> findAll();
    User save(User user);
    void deleteById(int userId);
}
```

The Service tier only ever depends on the interface, never on `UserDAOImpl` directly. This means the SQL is in exactly one place per entity, and a Service class can be unit-tested against a stub DAO instead of a real database — every one of my Service-layer tests does exactly this (see Task C).

2.2 Singleton

`DBConnectionManager` is the single point that loads `db.properties` and registers the JDBC driver:

```
public final class DBConnectionManager {

    private static final DBConnectionManager INSTANCE = new DBConnectionManager();

    private final String url;
    private final String username;
    private final String password;

    private DBConnectionManager() {
        Properties props = new Properties();
        try (InputStream in = getClass().getClassLoader().getResourceAsStream("db.properties")) {
            if (in == null) {
                throw new IllegalStateException(
                        "db.properties not found on the classpath. Copy "
                        + "src/main/resources/db.properties.example to "
                        + "src/main/resources/db.properties and fill in your local MySQL credentials.");
            }
            props.load(in);
        } catch (IOException e) {
            throw new IllegalStateException("Failed to read db.properties", e);
        }

        this.url = props.getProperty("db.url");
        this.username = props.getProperty("db.username");
        this.password = props.getProperty("db.password");

        try {
            Class.forName(props.getProperty("db.driver"));
        } catch (ClassNotFoundException e) {
            throw new IllegalStateException("MySQL JDBC driver not found on the classpath", e);
        }
    }

    public static DBConnectionManager getInstance() {
        return INSTANCE;
    }

    public Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, username, password);
    }
}
```

Without this, every one of the eight DAO implementations would load the same properties file and register the same driver independently, which is both wasteful and a config value that could quietly drift between classes. **Honest limitation**: `getConnection()` opens a new physical connection every call rather than pooling connections (e.g. with HikariCP). For a coursework system with one user at a time this is not a real problem, but it would not scale to many concurrent staff without adding a connection pool behind this same Singleton — which is a small, contained change precisely because everything already goes through this one class.

2.3 Strategy

`NotificationService` (what to send, and when) depends only on an interface, not on a specific delivery mechanism:

```
public interface NotificationChannel {

  /** @return true if delivery succeeded (or was accepted for delivery) */
  boolean send(String recipient, String message);

  String name();
}
```

`SmsNotificationChannel` is the one implementation actually wired in today. Swapping in email or push notifications later means writing one new class and changing a single constructor call — nothing in the booking, cancellation, or reschedule code would need to change. I also made `NotificationService` fail safe by design: if the channel throws, the failure is logged and swallowed rather than breaking the appointment action it was attached to — a text message not going out should never cost a patient their booking.

Table 2: Design patterns implemented.

Pattern	Where	Problem it solve
DAO	Every entity (`UserDAO`, `AppointmentDAO`, etc.)	Keeps SQL out of the Service tier; one place to change per entity
Singleton	`DBConnectionManager`	One shared point of DB configuration instead of eight independent copies
Strategy	`NotificationChannel` / `SmsNotificationChannel`	Lets the delivery mechanism change without touching the booking logic

Critical evaluation. I considered Factory for building a `Bill`, but decided against it — a bill only ever needs one constructor call with the treatment cost and the flat consultation fee, so a factory would have added a layer of indirection with nothing to actually decide between. I also considered turning the wait/skip delay decision into a second Strategy (it is a genuine candidate — a dentist-caused delay and a patient Wait/Skip decision are two different algorithms for the

same problem, "what happens to the rest of the day"), but I left it as a plain enum and `switch` in `AppointmentService`, since there are exactly two clearly-bounded cases and no expectation of a third. I would rather note this as a considered-and-rejected pattern than claim more patterns than the code actually uses.

3. COMPLEX FUNCTIONALITY — SMS NOTIFICATIONS AND AN AUDIT TRAIL

Booking, cancelling, and rescheduling an appointment all trigger an SMS notification to the patient's contact number, through the Strategy pattern above. Every attempted notification is also written to a `notifications` table, so staff have a real, queryable record of what was (attempted to be) sent, not just a line in a log file.

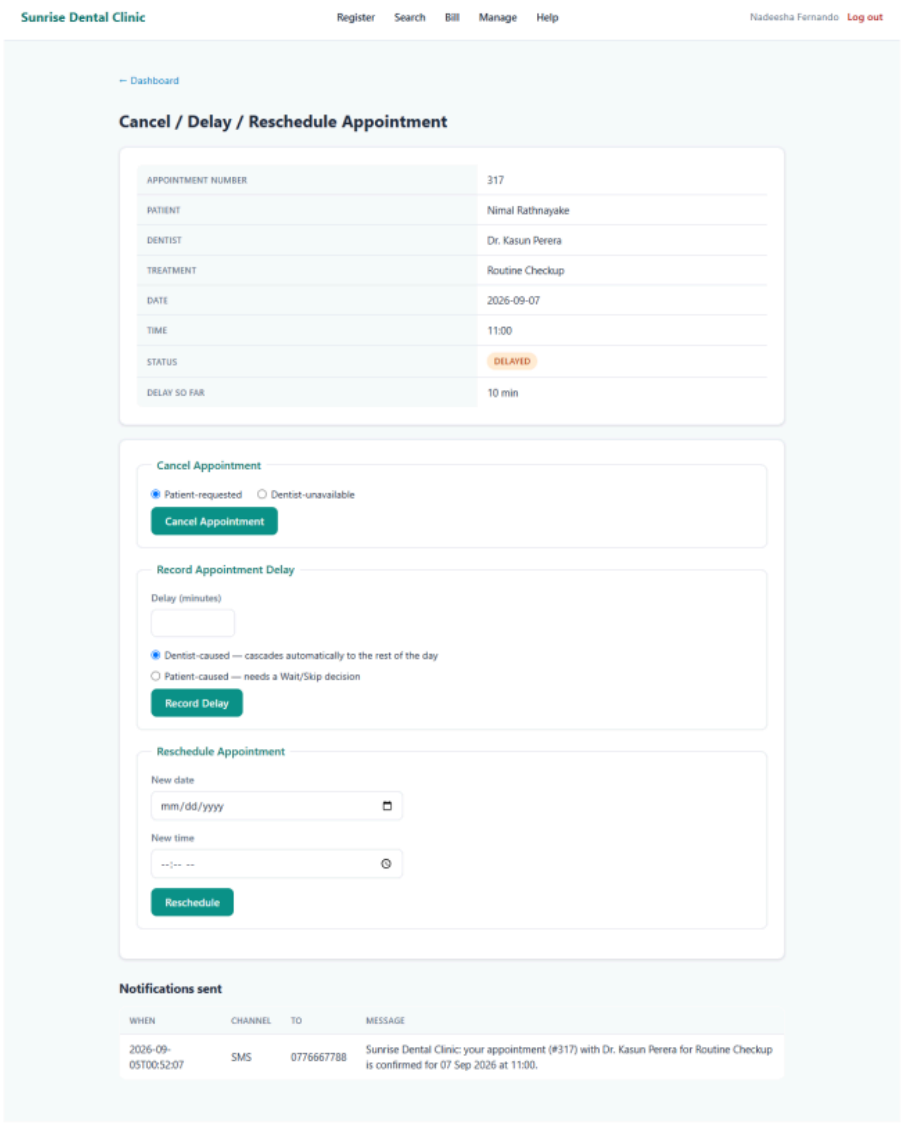

Figure 13: The notifications audit trail for an appointment, showing a real stored record.

Honest limitation. `SmsNotificationChannel` does not call a real SMS gateway (e.g. Twilio) — that needs a paid account and credentials this project does not have. What is real: the message is built correctly, logged, and recorded in the database exactly as it would be with a live gateway; only the actual carrier hop is simulated. I also found, while testing, that a dentist-caused or patient-caused delay never triggers a notification at all — only Cancel and Reschedule do — even though a delay is arguably the change a patient most needs to know about. I left this open deliberately rather than guess at an answer, because it raises a real design question (should a cascaded delay notify every affected patient, or only the one who triggered it, and what should the message say for each) that deserves a considered decision, not a rushed one.

4. VALIDATED USER INPUT

Every form is validated on the server, not just in the browser, because a user can always bypass client-side JavaScript. A few examples, with the exact wording the system shows:

Table 3: Server-side validation rules.

Field/Action	Rule	Message shown
Patient name, address, contact number	Must not be blank	"Patient name, address and contact number are all required."
Contact number	Must match a Sri Lankan mobile number pattern	"Contact number must be a valid Sri Lankan number, e.g. 0771234567 or +94771234567."
Appointment date	Cannot be in the past	"Appointment date cannot be in the past."
New/updated appointment slot	No overlapping booking for that dentist	"[Dentist] is not available on [date] at [time] — please choose a different slot."
Staff account fields	Must not be blank	"Username, password, full name and role are all required."
Staff account password	At least 6 characters	"Password must be at least 6 characters."
Daily appointment limit	Must be greater than zero	"Daily appointment limit must be greater than zero."
Availability block	End time must be after start time	"The end of an unavailable period must be after its start."

The double-booking check is enforced twice, on purpose: once in `AppointmentService` before the insert is attempted (a fast, friendly error for the normal case), and once again inside the database itself with a trigger (Section 5) that physically cannot be bypassed, even if a future change adds a second way to insert an appointment that skips the Service tier. I would rather have one redundant check than one gap.

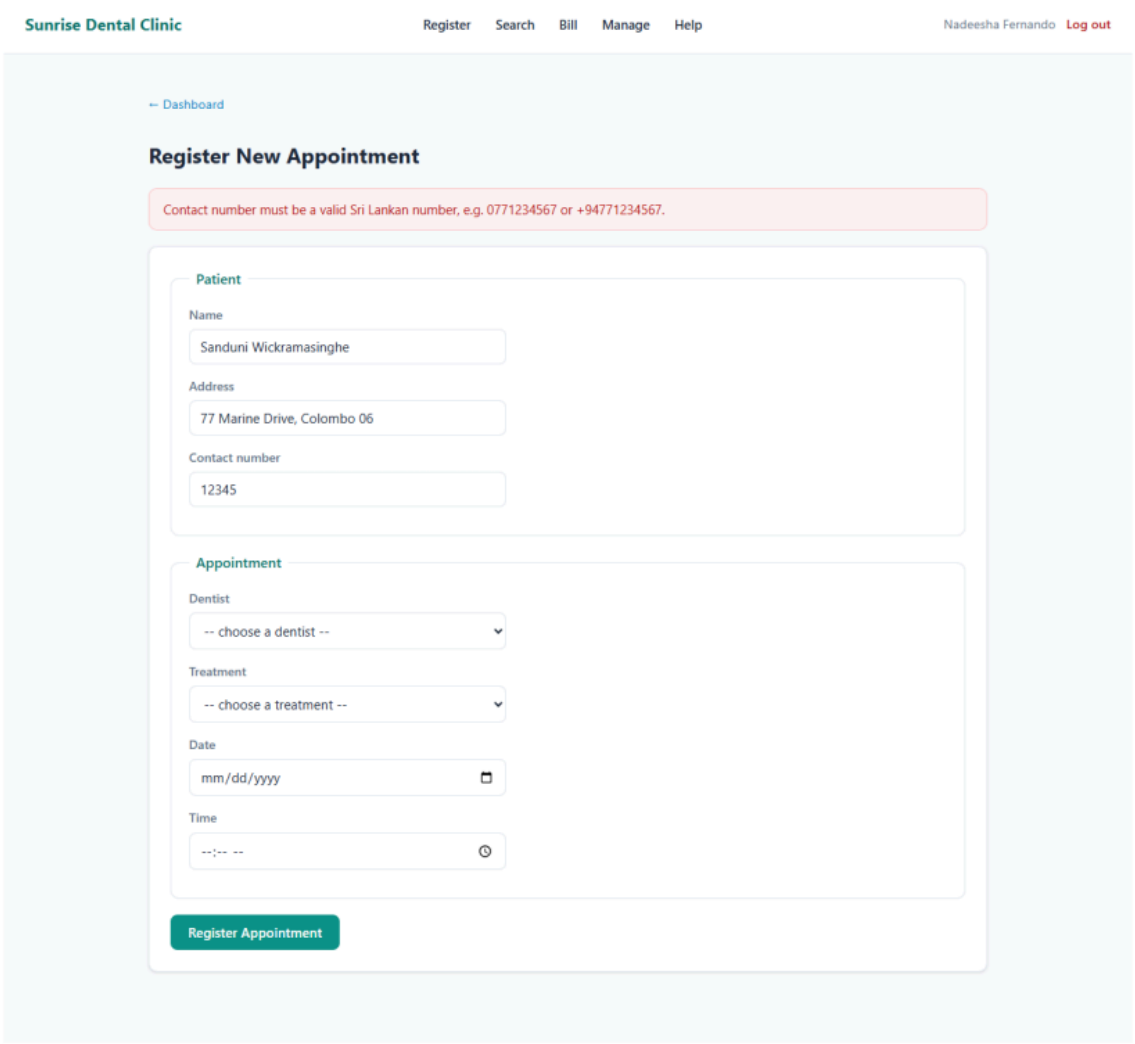

Figure 14: Server-side validation rejecting an invalid contact number.

I also escape every piece of user-entered text before it is displayed back (JSTL's `fn:escapeXml()`, used across every JSP that echoes a name, address, or contact number) so a patient name containing HTML or script characters cannot be rendered as markup on someone else's screen.

5. DATABASE DESIGN AND ADVANCED FEATURES

The database has eight tables: `users` (with a `role` column separating Receptionist/Administrator/Dentist), `dentists` (extends a user with capacity/specialisation fields), `availability_blocks`, `patients`, `treatments`, `appointments` (the central table, holding status, delay minutes, and change reason), `bills`, and `clinic_settings` (a single-row table for the flat consultation fee), plus notifications for the audit trail above. Foreign keys cascade delete where a child record has no meaning without its parent (a bill without its appointment, an availability block without its dentist).

Beyond the schema itself, I used three MySQL features that go past plain `SELECT`/`INSERT`:

Two triggers stop a double booking at the database level, regardless of which application code path tries to insert or update an appointment:

```
CREATE TRIGGER trg_appointments_no_double_booking_insert
BEFORE INSERT ON appointments
FOR EACH ROW
BEGIN
  DECLARE v_conflicts INT;
  IF NEW.status <> 'CANCELLED' THEN
    SELECT COUNT(*) INTO v_conflicts
    FROM appointments a
    JOIN treatments t ON t.treatment_id = a.treatment_id
    JOIN treatments nt ON nt.treatment_id = NEW.treatment_id
    WHERE a.dentist_id = NEW.dentist_id
      AND a.appointment_date = NEW.appointment_date
      AND a.status <> 'CANCELLED'
      AND a.appointment_time < ADDTIME(NEW.appointment_time,
SEC_TO_TIME(nt.duration_minutes * 60))
      AND NEW.appointment_time < ADDTIME(a.appointment_time,
SEC_TO_TIME(t.duration_minutes * 60));

    IF v_conflicts > 0 THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Double-booking guard: this dentist already has an overlapping
appointment at that time.';
    END IF;
  END IF;
END$$
```

A matching `BEFORE UPDATE` trigger covers rescheduling, excluding the row's own current slot so moving an appointment to the same time it already has does not falsely reject itself.

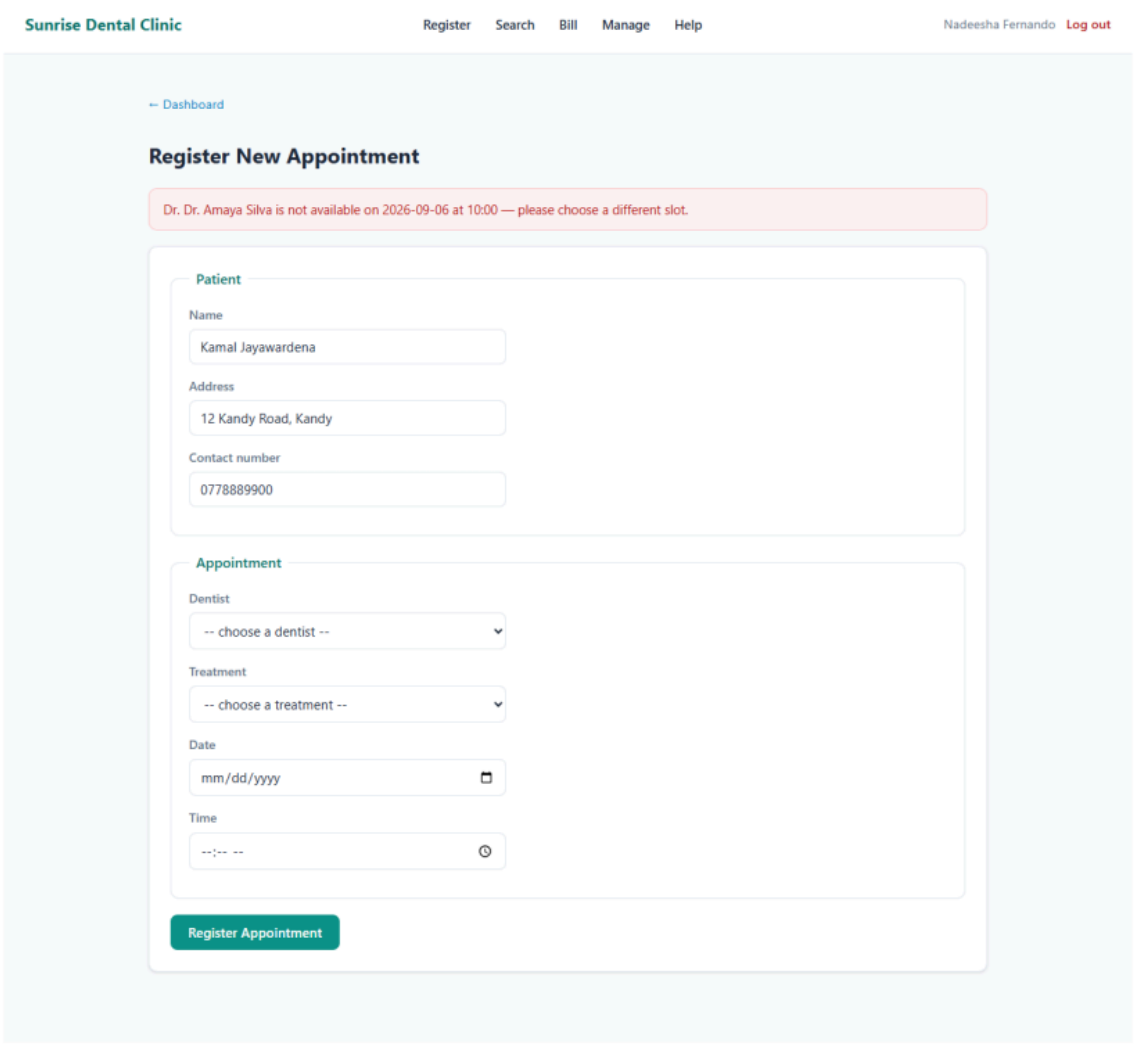

Figure 15: A booking rejected because it clashes with an existing appointment — enforced by the database trigger, not just the Service-tier check.

A function calculates the bill total inside the database, so the stored figure can never drift from the formula:

```
CREATE FUNCTION fn_bill_total(p_treatment_cost DECIMAL(10,2), p_consultation_fee
DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  RETURN p_treatment_cost + p_consultation_fee;
END$$
```

A stored procedure works out a dentist's revenue for a given day, called from Java through a `CallableStatement`:

```
CREATE PROCEDURE sp_dentist_daily_revenue(IN p_dentist_id INT, IN p_date DATE, OUT p_revenue
DECIMAL(10,2))
```

```
BEGIN
  SELECT COALESCE(SUM(b.total_amount), 0) INTO p_revenue
  FROM bills b
  JOIN appointments a ON a.appointment_number = b.appointment_number
  WHERE a.dentist_id = p_dentist_id AND a.appointment_date = p_date;
END$$
```

This is used directly by the reports feature below.

6. REPORTS FOR DECISION-MAKING

The View Reports screen (Administrator only) gives three pieces of information a clinic manager would actually use day to day: how many appointments are currently in each status clinic-wide, total revenue billed to date, and — for a chosen date — how many appointments and how much revenue each dentist is carrying, with the per-dentist revenue figure coming from the stored procedure above rather than a second plain query. This lets an administrator see, for example, whether one dentist is consistently busier (and earning more) than another, which is the kind of thing that should actually inform a staffing decision, not just a number for its own sake.

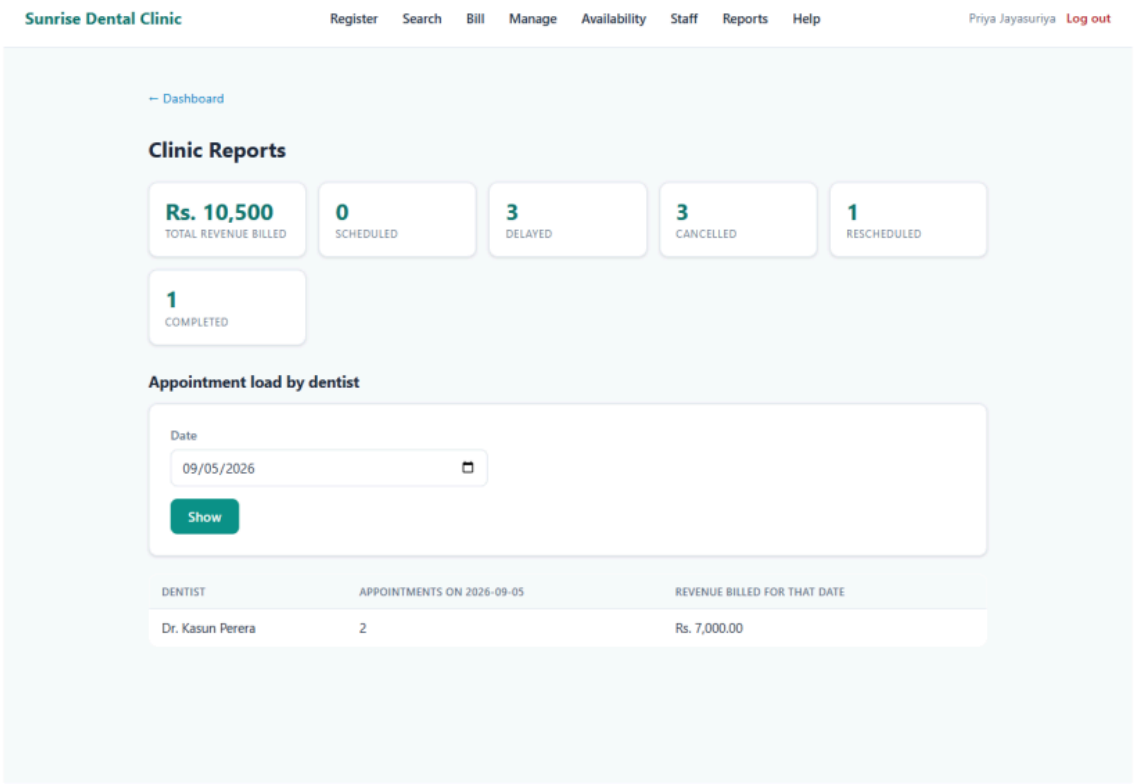

Figure 16: The reports screen, showing clinic-wide totals and a per-dentist breakdown driven by the stored procedure.

7. SESSIONS, COOKIES, AND ACCESS CONTROL

Logging in creates an `HttpSession` and stores three attributes on it: the logged-in user object, and two booleans (`isAdmin`, `isDentist`) so later requests do not need to re-derive the role from the database on every page. Straight after authentication succeeds, I call `request.changeSessionId()` before setting any of these attributes — this defends against session fixation, where an attacker who already knows a victim's (pre-login) session ID could otherwise hijack the session the moment it becomes privileged.

Role checks are centralised in one small class (`RoleGuard`) rather than repeated in every servlet, after I found — by testing as a Dentist — that the navigation menu correctly hid Register/Search/Bill for that role, but nothing actually stopped a Dentist from reaching those pages directly by typing the URL. `RoleGuard` fixes this at the server side for every route that needs it: Administrator-only actions (staff accounts, reports), Dentist-or-Administrator actions (daily limit, availability), and receptionist-or-administrator-only actions (register, search, bill — a use case diagram never gives a Dentist these), so the actual access control matches the use case diagram exactly, not just the menu the user happens to see.

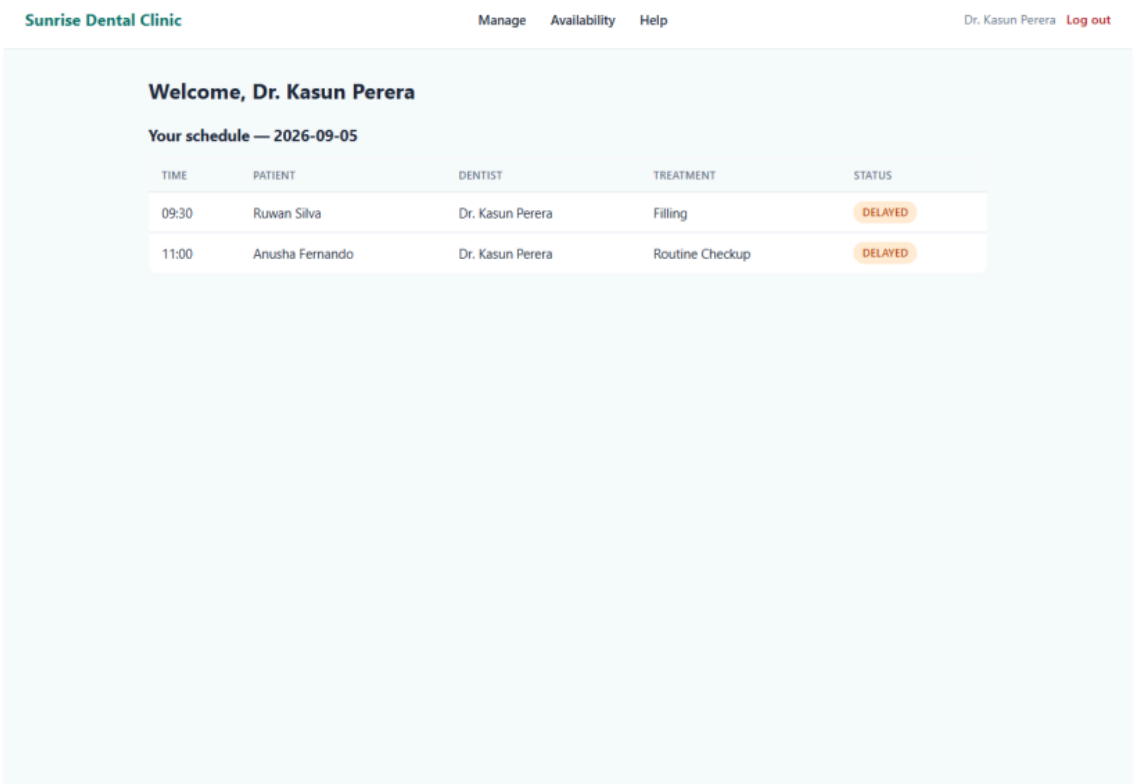

Figure 17: A Dentist attempting to reach the registration form directly by URL is redirected to the dashboard, not shown the form.

8. SUMMARY AND CRITICAL EVALUATION

The finished system is a genuine three-tier, client-server web application: a browser talking over HTTP to Servlets, which call Services, which call DAOs, which are the only code allowed to touch the database. It uses three design patterns for real reasons rather than for their own sake, has server-side validation on every form, two levels of database-enforced integrity beyond the schema (triggers, a function, and a stored procedure), a working notification feature with a real audit trail, decision-useful reports, and role-based access control enforced on the server, not just hidden in the UI.

The honest limitations I would flag myself: no connection pooling behind the Singleton database manager; SMS delivery is simulated rather than sent through a real gateway; a delay does not currently trigger a notification, which I left open as a design question rather than guessing at one; a small cosmetic bug where a dentist's name is shown as "Dr. Dr. [name]" in two unavailability messages, because their stored name already includes the title and the message code adds a second one; and the availability table shows a raw ISO date-time (`2026-09-20T13:00`) rather than a formatted one. None of these affect correctness of the actual business rules, but I would rather list them here than have them look like something I missed.

TASK C – TESTING: TEST PLAN, TEST-DRIVEN DEVELOPMENT, AND TEST AUTOMATION

INTRODUCTION

Testing on this project happened at two different levels, on purpose, because they catch different kinds of mistakes. Automated JUnit tests check that the business rules and the database behave correctly, every time, and run automatically on every push through the CI workflow from Task D. A separate, scripted end-to-end walkthrough of the actual running application checks the things a unit test cannot see at all — whether a real page renders correctly, whether an error message actually makes sense to a receptionist reading it, whether a security rule is enforced the whole way through a real HTTP request. Both layers found real defects. This section explains the rationale for that approach, how test-driven development was actually used (not just claimed), the test data behind it, the resulting test plan, and how it was applied.

1. TEST RATIONALE

I put the JUnit suite at the Service and DAO layers rather than only testing the UI, because that is where the actual business rules live — availability checks, delay cascades, billing totals, database triggers — and a bug there is a bug in every screen that touches it. Testing at this layer is also fast and fully automatable: it runs in a few seconds, needs no browser, and is exactly the kind of check that belongs in the CI pipeline on every single push (Task D's `ci.yml`), so a broken rule is caught before it is ever demonstrated to anyone.

I deliberately did not try to automate the browser-level walkthrough into CI as a permanent job. A scripted browser test needs a fully deployed Tomcat and MySQL instance behind it, which is a heavier, slower, more fragile thing to keep running on every push than a project this size justifies. Instead, I used it as a periodic, deliberate acceptance check — run in full once the implementation for a set of features was believed complete, specifically to catch the class of bug that only shows up once everything is wired together (an encoding problem in the actual HTML output, a wrong word in an error message, a security rule that exists in the code but was never actually hooked up to a route). Both kinds of testing found real problems the other layer could not have caught, which is covered in Section 5.

2. TEST-DRIVEN DEVELOPMENT — HOW IT WAS ACTUALLY USED

The stated approach from the start of the project was to write tests test-first where practical. I want to report honestly how consistently that was actually followed, based on checking real commit history rather than repeating what any one test's own comment claims about itself.

Followed most consistently for early, core business-rule tests. `AppointmentServiceTest` — the largest test class, with 14 assertions covering the three-part availability check and the cancel/delay/reschedule flows — was written against hand-written test doubles for the DAOs rather than a real database, which is exactly the shape a test written to drive out business logic before wiring up persistence would take.

Not applied uniformly, and I looked at why. Comparing when each test file was first committed against when the code it tests was first committed shows three different real patterns, not one consistent practice:

- A tight, same-session loop (test committed 13–39 seconds after the code) for `AppointmentDAOImplTest`, `AppointmentServiceTest`, and `BillServiceTest` — writing the code, then immediately confirming it with a test, rather than the test existing first and failing.
- Genuinely test-after, sometimes by hours: `UserDAOImplTest` and `UserServiceTest` were not added until roughly four hours after `UserDAOImpl`/`UserService` themselves — code that had already been in real use through `LoginServlet` that whole time. `BillDAOImplTest` was added about two and a half hours after `BillDAOImpl`, once the database function it actually exercises (`fn_bill_total`) existed. This makes sense: a DAO-level test needs a real schema and real data in front of it, so testing the database layer strictly test-first is often impractical — the schema has to exist before a test against it can even fail meaningfully.
- Bundled under time pressure: `NotificationServiceTest`, `DentistServiceTest`, and `ReportServiceTest` were each committed in the same commit as their implementation, so which was actually written first is not something the history can show either way.

`BillTest` — the very first test class in the project — carries a comment describing it as written deliberately test-first. The commit timestamps show `Bill.java` was committed about 11 minutes before `BillTest.java`, which does not clearly support that specific claim (the two could have been edited in either order inside that window, and git cannot see that). I would rather report this honestly than repeat the comment's claim uncritically: TDD was the genuine stated intent from the first test class onward, it was followed most closely for the earliest core business-rule logic, and it gave way to test-after and bundled patterns once real persistence and time pressure entered the picture. I think that is a more useful account than claiming strict red-green-refactor discipline throughout, and it matches a pattern I would expect on a real project — the parts of a system with the clearest, most self-contained rules are the easiest to drive with a failing test first, and the parts that depend on infrastructure are the hardest.

3. TEST DATA

Two different kinds of test data feed the suite, deliberately different in shape:

Realistic seed data (`seed-data.sql`) — four staff accounts (a receptionist, an administrator, and two dentists, with real BCrypt-hashed passwords rather than plain text), five treatments at different price points and durations, five patients, and a spread of appointments across yesterday, today, and next week, across both dentists, in a mix of statuses (completed, cancelled, scheduled, delayed, rescheduled) — including one already-billed appointment so the reports feature has real revenue to show. This exists specifically so every screen — a dashboard's "today's schedule," a dentist's own view, the reports page's status counts and per-dentist load

— has real, varied data to display rather than an empty table, and every appointment's time is deliberately spaced so the seed data itself never trips the double-booking trigger.

Isolated, self-contained fixtures for tests that touch the database. `DoubleBookingTriggerTest`, for example, creates and tears down its own dentist, patient, treatment, and appointment rows in `@BeforeEach`/`@AfterEach`, using a test date two years in the future specifically so it can never collide with real seed or demonstration data. This keeps the automated suite repeatable — it can run against a database that already has real data in it (as CI's does, loaded from the same `schema.sql`) without depending on, or corrupting, that data.

A third, separate, deliberately temporary data change supported only the UI walkthrough: the four seeded accounts' passwords have no recorded plaintext anywhere (the hashes were generated once and never written down, by design), so running a real browser login required resetting them to a known value in the local development database only — never touching `seed-data.sql` or anything committed to the repository.

4. THE AUTOMATED TEST SUITE

Table 5: The 12 JUnit test classes, what they exercise, and how many checks each contains.

Test class	Layer	Tests	What it proves
`BillTest`	Domain model	2	Bill total = treatment cost + consultation fee
`AppointmentServiceTest`	Service (test doubles)	14	The three-part availability check; cancel, delay (dentist/patient, wait/skip), and reschedule flows
`BillServiceTest`	Service (test doubles)	2	Bill generation delegates correctly to the DAO/database
`UserServiceTest`	Service (test doubles)	6	Login verification (BCrypt), password hashing on account creation
`DentistServiceTest`	Service (test doubles)	2	Daily limit and availability-block updates
`NotificationServiceTest`	Service (stub channel/DAO)	3	Notifications reach the right recipient, are persisted, and a channel failure never breaks the calling action
`ReportServiceTest`	Service (stub DAO)	3	Reporting queries delegate correctly
`AppointmentDAOImplTest`	DAO (real database)	6	Real SQL for saving/finding appointments, including reschedule's self-exclusion logic

`UserDAOImplTest`	DAO (real database)	4	Real SQL for finding/saving/removing users
`BillDAOImplTest`	DAO (real database)	1	The `fn_bill_total()` database function is actually called and correct
`DoubleBookingTriggerTest`	Database (raw JDBC)	2	The database-level trigger blocks an overlapping raw insert and allows a back-to-back one
`DBConnectionManagerTest`	Infrastructure	2	The Singleton returns a working, usable connection

47 tests in total, run automatically with `mvn test`, and on every push/pull request to `main` through the GitHub Actions workflow described in Task D — the same MySQL 8 service container and the same `schema.sql` back both the DAO-level tests above and CI itself, so "passes in CI" and "passes locally" are never testing against two different databases.

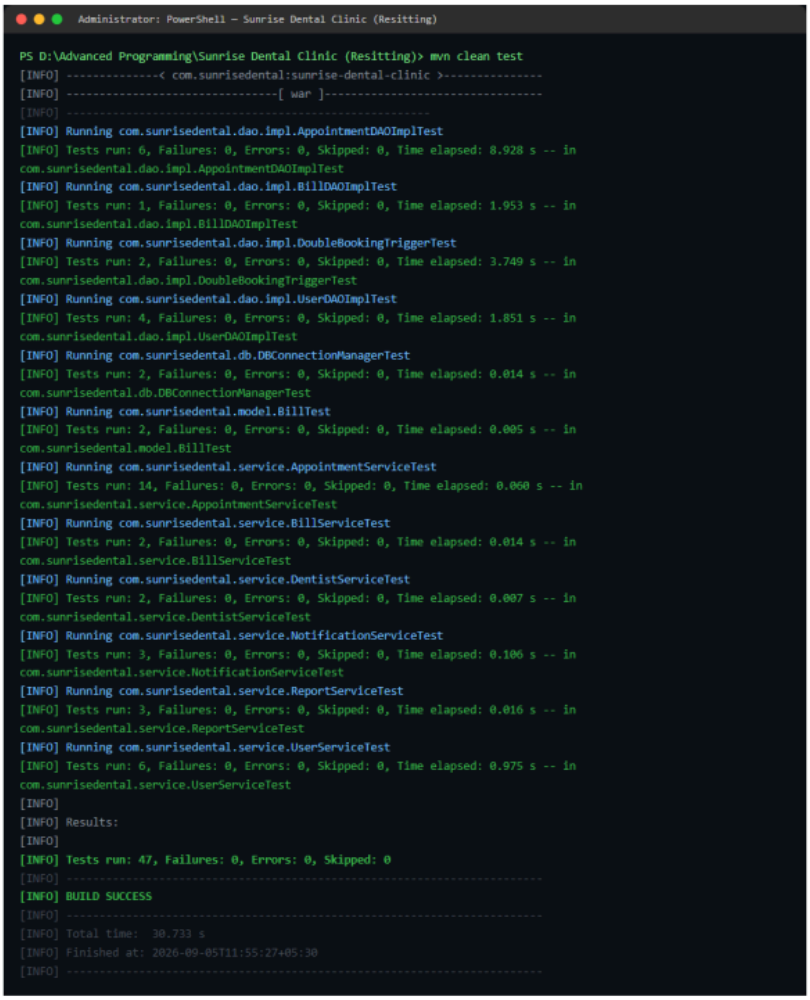

Figure 21: A fresh `mvn clean test` run — 47 tests, 0 failures, 0 errors, 0 skipped, BUILD SUCCESS.

5. TRACEABILITY — REQUIREMENT TO DESIGN TO TEST

Each decision number below is a short reference to a specific design choice explained in Task A/B — see the Design decisions.md file for a one-line summary of each one, so a decision can be looked up on its own without paging back through the earlier tasks.

Table 6: How each requirement is backed by a design decision and confirmed by a test or a documented screenshot.

Requirement / use case	Design decision(s)	Confirmed by
Login	Decisions 1, 16	`UserServiceTest` (login/BCrypt); UI walkthrough screenshots 2–5
Register New Appointment (+ availability check, + new patient)	Decisions 2, 3, 24, 28	`AppointmentServiceTest`, `AppointmentDAOImplTest`, `DoubleBookingTriggerTest`; screenshots 7–12
Display Appointment Details (Search)	Decision 6	`AppointmentDAOImplTest`; screenshots 13–14
Calculate & Print Bill	Decisions 26, 27	`BillTest`, `BillServiceTest`, `BillDAOImplTest`; screenshots 15–16
Help	— (static content)	Screenshot 24
Exit / Logout	Session invalidation	Not unit tested (nothing to assert beyond session teardown); screenshots 25, 42
Cancel Appointment	Decisions 6, 7, 8, 30	`AppointmentServiceTest`; screenshot 18
Record Appointment Delay (dentist/patient, wait/skip)	Decisions 9, 10, 11, 17, 18	`AppointmentServiceTest`; screenshots 19–21
Reschedule Appointment	Decisions 12, 28, 29	`AppointmentServiceTest`; screenshot 22
Manage Staff Accounts (Admin)	Decision 5	`UserServiceTest`, `UserDAOImplTest`; screenshots 27–31
View Reports (Admin)	Decision 5	`ReportServiceTest`; screenshot 35
Set Daily Limit / Availability (Dentist, Admin)	Decisions 13, 14	`DentistServiceTest`; screenshots 32–34, 41
SMS notifications (complex functionality)	Decision 31	`NotificationServiceTest`; screenshot 23
Role-based access control (Dentist boundary)	Decision 30	No dedicated `RoleGuard` unit test — confirmed only through the UI walkthrough; screenshots 36–41

Double-booking prevented at the database level	`schema.sql` triggers	`DoubleBookingTriggerTest`; screenshot 11

Building this table honestly surfaced a real gap rather than papering over one: `RoleGuard`, the class enforcing every role boundary in the system, has no dedicated JUnit test of its own — it is only ever confirmed indirectly, through the end-to-end walkthrough. That is a genuine hole in the automated suite, not a design problem, and I have said so rather than leaving it to be found.

6. APPLYING THE TEST PLAN — THE END-TO-END WALKTHROUGH

Once the automated suite was passing, I ran a full scripted walkthrough of the actual deployed application (real Chrome, driven against the real running Tomcat and MySQL, no mocks) covering every brief-required function and every additional use case, across all three roles. Every screenshot is numbered, and the full write-up — a test-ID table of feature, input, expected result, actual result taken directly from the live page, and verdict — lives in the project's `test-screenshots/TEST-LOG.md`.

The walkthrough found two real defects, both fixed and re-verified before the run was called complete:

A JSP source-encoding bug. No page declared its character encoding, so every em dash on the shared page layout rendered as mangled characters. The first fix attempt (adding the encoding declaration to only the shared header fragment) was tested and provably did not work, because the page including it had already been decoded incorrectly by the time that fragment ran. The real fix — declaring the encoding on every page Tomcat compiles directly — was then verified by re-running the same test and checking the same pages.

A misleading error message. Attempting to remove a staff account that the database correctly refuses to delete (a dentist who still has appointments) originally showed "That username is already taken" — wrong on its face for a removal, not just unhelpful. Caught because an early version of the test script picked the wrong row to click, which is itself a small but real finding about relying on table row order. Fixed by making the error message depend on which action actually failed.

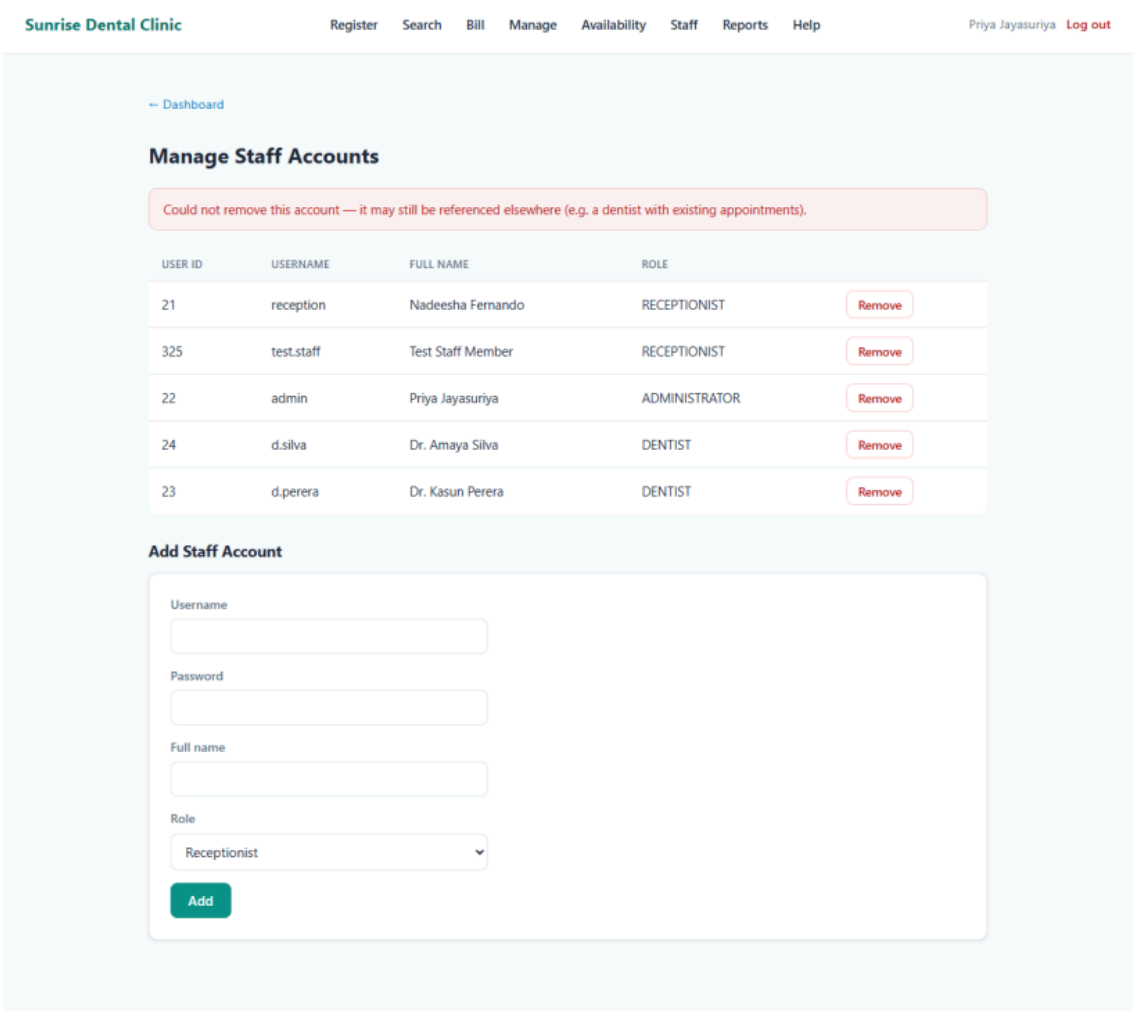

Figure 22: The corrected error message shown after the fix, on a deliberate repeat of the exact scenario that first exposed the bug.

One further defect was found and deliberately left unfixed: a delay never triggers a patient notification, while cancel and reschedule both do. This was not treated as a quick patch, because it raises a genuine design question — should every affected appointment in a cascade be notified, or only the one that triggered it, and what should the message say — that deserved a considered answer rather than a guess made under test-time pressure. It is documented as an open decision rather than silently left for someone else to notice.

7. SUMMARY AND CRITICAL EVALUATION

Every one of the 47 automated tests passes, and the final full run of the 42-screenshot walkthrough passed every case as well, against a build that had already absorbed both fixes above. I think the most useful lesson from this project's testing is that the two layers genuinely caught different classes of bug: neither the encoding problem nor the wrong error message would ever have been caught by a JUnit test, because both are about what actually reaches a real browser, not about whether a Java method returns the right value. Equally, the database

trigger and the availability-check logic are exactly the kind of rule that is far cheaper and far more reliable to check with an automated test run on every push than by remembering to click through it by hand each time.

The TDD account in Section 2 is the other honest lesson here: the intention was there from the start, and it held up best where the rules were self-contained and didn't depend on a database existing yet; it slipped under real time pressure and once persistence entered the picture, which I think is a realistic account of how test-first discipline actually survives contact with a real deadline, rather than a claim that everything was done by the book throughout.

TASK D – GIT AND GITHUB: VERSION CONTROL AND WORKFLOW

INTRODUCTION

This part covers how I used Git and GitHub to manage the project, not just as a backup, but as a real record of how the system was built — one change at a time, with a working build checked automatically on every push. I kept the repository focused on exactly what is needed to build, test, and run the submitted system, and I explain that decision below rather than leaving it unexplained.

1. REPOSITORY SETUP AND ACCESS

The project is hosted as a public GitHub repository, created from the start of development rather than added at the end. Being public means anyone can view the code and commit history and clone it read-only; write access (the ability to push a change) is restricted to my own account, which is the right level of access control for a solo coursework project — nobody else needs to write to it, but a marker or anyone else needs to be able to read it without asking for permission first.

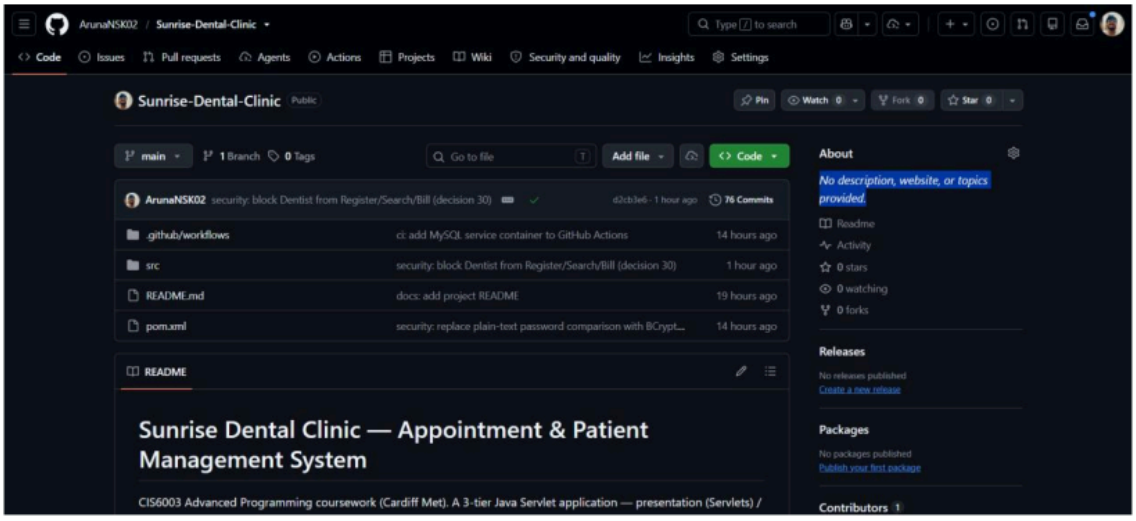

Figure 18: The public GitHub repository, showing its file structure and commit history.

I deliberately kept the repository's tracked contents to only what a marker actually needs to build and run the submitted system: the Java source, the Maven build file, the CI workflow, the schema, and the README. Local planning notes, the assessment brief PDF, and my own working scripts are excluded through `.gitignore` — because they are development scaffolding, not part of the deliverable, and a repository that is easy to read matters as much as one that is complete. The same file also keeps real database credentials out of version control (`db.properties` is ignored; a `db.properties.example` with placeholder values is committed instead, so anyone cloning the repository can see exactly what to fill in without my own local password ever being in the history).

2. VERSION CONTROL TECHNIQUES

I used a small number of deliberate techniques consistently, rather than one big commit at the end:

Trunk-based development. Everything is committed directly to `main`, with no long-lived feature branches. For a solo project of this size, a branching strategy built for coordinating multiple people would add process without adding safety — the discipline that actually matters here is keeping each individual commit small and correct, which the next two points cover.

One commit per logical change. Each commit is scoped to one complete slice of work — one use case, one bug fix, one refactor — rather than batching unrelated changes together. This means the history itself is a readable log of decisions: anyone (including me, checking back later) can see exactly when and why a particular rule or fix was introduced, and can revert one change without dragging in unrelated ones.

A consistent commit message convention. Every message starts with a short prefix describing the kind of change, followed by a plain description, and the body (where needed) explains the reasoning, not just the mechanics.

Table 4: Commit prefixes used and what they mean.

Prefix	Used for
`feat:`	A new use case or feature
`feat:`	A new use case or feature
`fix:`	Correcting a bug
`security:`	Closing an access-control or validation gap
`db:`	Schema, trigger, function, or seed-data changes
`web:`	Presentation-tier changes (JSPs, shared layout, servlets)
`docs:`	Documentation-only changes
`chore:`	Build/config housekeeping (e.g. `.gitignore`)

I found this useful in practice, not just as a formality — being able to scan the commit list and immediately tell which changes were feature work versus fixes versus security corrections made it much easier to check my own progress against the brief's six required functions plus the extra use cases I added.

3. CONTINUOUS INTEGRATION WORKFLOW

The repository has a GitHub Actions workflow that builds the project and runs the full automated test suite on every push and every pull request to `main` — this was set up early in the project, not bolted on at the end, so I always had a check on whether the current state of `main` actually still worked.

The workflow does more than a bare `mvn package`: it starts a real MySQL 8 service container inside the CI run itself, waits for it to be ready, and loads the exact same `schema.sql` file the project uses locally — so the database CI tests against is built from the same source of truth as my own development database, rather than a hand-maintained copy that could quietly drift out of sync. This matters because a meaningful share of the test suite is real DAO-level integration tests that hit an actual database (checking the double-booking trigger, the bill-total function, and the stored procedure genuinely work, not just that the Java code compiles), so a CI job with no real database behind it would not actually prove very much.

```yaml
name: Build and Test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: sunrise_dental
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h localhost"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=10

    steps:
      - uses: actions/checkout@v4
```

```yaml
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'microsoft'
    cache: maven

- name: Ensure mysql client is available
  run: |
    if ! command -v mysql >/dev/null; then
      sudo apt-get update -yqq
      sudo apt-get install -yqq mysql-client
    fi

- name: Wait for MySQL and load schema.sql
  run: |
    for i in $(seq 1 30); do
      mysqladmin ping -h127.0.0.1 -P3306 -uroot -proot --silent && break
      sleep 2
    done
    mysql -h127.0.0.1 -P3306 -uroot -proot sunrise_dental < src/main/resources/schema.sql

- name: Create db.properties (CI credentials — never committed, see .gitignore)
  run: |
    cat > src/main/resources/db.properties << 'EOF'
    db.url=jdbc:mysql://127.0.0.1:3306/sunrise_dental?useSSL=false&serverTimezone=UTC
    db.driver=com.mysql.cj.jdbc.Driver
    db.username=root
    db.password=root
    EOF

- name: Build and run tests
  run: mvn -B package
```

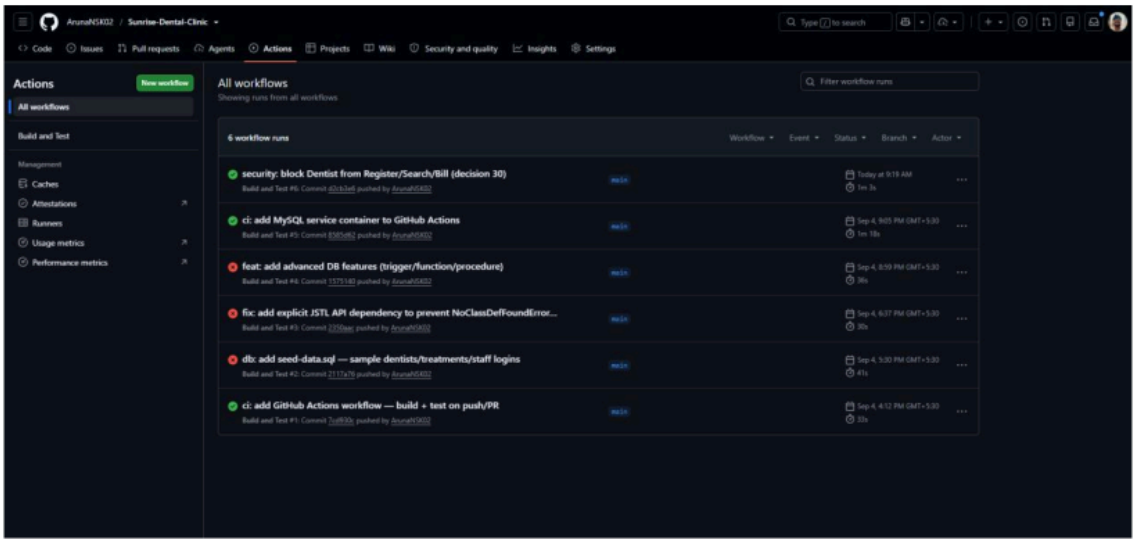

Figure 19: The GitHub Actions run history, showing the pipeline in action over time.

This history is worth showing in full rather than just a single passing run, because it demonstrates the pipeline actually catching a real problem rather than just rubber-stamping the build. The three earliest runs failed — "db: add seed-data.sql", "fix: add explicit JSTL API dependency…", and "feat: add advanced DB features (trigger/function/procedure)" — because at that point the workflow had no database to test against, so anything touching the database had nothing to connect to. The very next commit, "ci: add MySQL service container to GitHub Actions", fixed it by adding the service container above, and every run since has passed. I see this failure-then-fix sequence as better evidence of a working CI setup than a single green tick would be, since it shows the checks were actually enforced, not decorative.

Honest limitation. This is a continuous integration workflow, not a continuous deployment one — it proves the build and tests pass, but it does not push a build anywhere automatically. I did not add a deployment step because there is no real deployment target for this coursework beyond a local Apache Tomcat instance (the brief specifies Tomcat, not a cloud host), and a "deploy" job with nowhere real to deploy to would just be for show. I see this as a considered scope decision, not something I ran out of time for.

4. DEPLOYMENT

Deployment for this project means: build the WAR with Maven, and deploy it to a local Apache Tomcat 11 instance backed by a local MySQL 8 database built from the same `schema.sql` CI uses. The README documents this exact sequence (create the database, load the schema, copy the example properties file and fill in local credentials, `mvn package`, drop the resulting WAR into Tomcat's `webapps` folder) so the steps are reproducible by anyone, not just remembered by me.

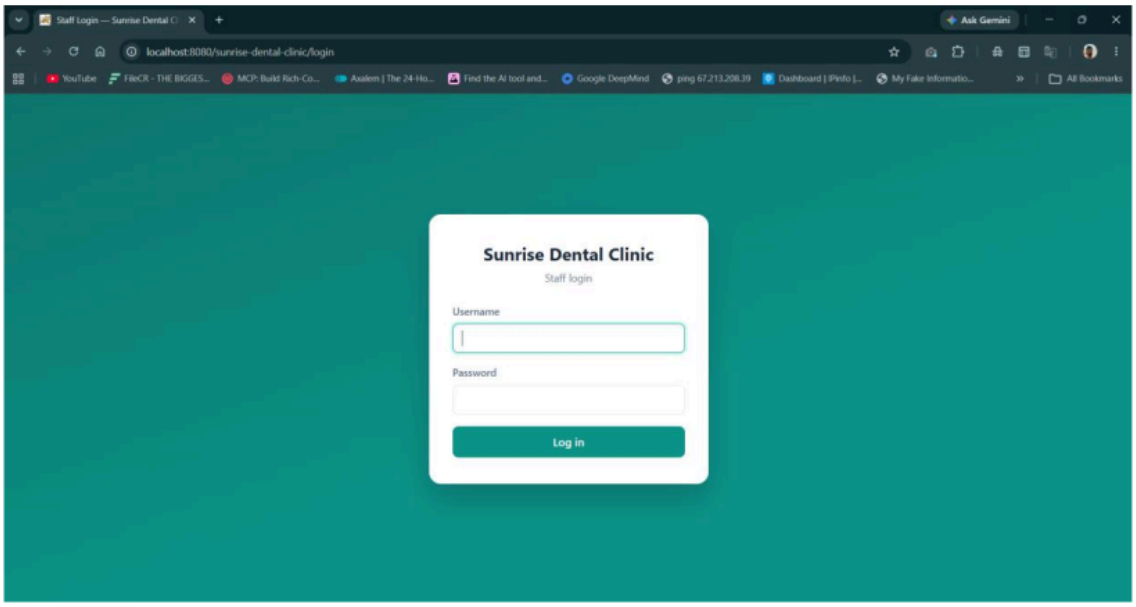

Figure 20: The latest version of the application, built from the current main branch, running on a local Tomcat instance.

5. DOCUMENTATION

The repository's `README.md` is the entry point for anyone landing on the project: what it is, the technology stack, the package structure, and exact steps to get it running locally. I kept it deliberately short and practical rather than duplicating the full design rationale (that level of detail belongs in the report, not the code repository).

Rather than linking out to a copy hosted somewhere else, I committed the final report itself into the repository and linked to it directly from the README, so the "report link within the documentation" the brief asks for is just a normal relative link to a file that is actually part of the repository, not a pointer to something external that could move or be taken down later.

6. SUMMARY AND CRITICAL EVALUATION

The repository shows a real, working version control process: a public repo with restricted write access, small and clearly labelled commits, a consistent naming convention, and a continuous integration workflow that actually exercises the database rather than just compiling the code. I think the strongest part of this is that the CI workflow and the local development setup are built from the exact same `schema.sql`, so "it works in CI" and "it works locally" are backed by the same database definition rather than two copies that could disagree.

The honest limitations: there is no continuous deployment step, since there is no real deployment target beyond a local Tomcat instance; there are no long-lived feature branches or pull requests, which is appropriate at this project's solo scale but would not be a sufficient version control strategy for a team; and there are no tagged releases marking specific milestones, which I would add if I were continuing this project past the coursework deadline,

so a specific submitted version could be pointed to directly by a tag rather than by commit hash alone.

REFERENCE LIST

- Alur, D., Crupi, J. and Malks, D. (2003) *Core J2EE Patterns: Best Practices and Design Strategies*. 2nd edn. Upper Saddle River: Prentice Hall.
- Beck, K. (2002) *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Eclipse Foundation (2022) *Jakarta Servlet Specification, Version 6.0*. Available at: https://jakarta.ee/specifications/servlet/6.0/ (Accessed: 5 September 2026).
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.
- GitHub, Inc. (2024) *GitHub Actions Documentation*. Available at: https://docs.github.com/actions (Accessed: 5 September 2026).
- Object Management Group (2017) *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Available at: https://www.omg.org/spec/UML/2.5.1/ (Accessed: 5 September 2026).
- Oracle Corporation (2024) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 5 September 2026).
- Provos, N. and Mazières, D. (1999) 'A Future-Adaptable Password Scheme', *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. Monterey, CA, 6–11 June.
- The JUnit Team (2024) *JUnit 5 User Guide*. Available at: https://junit.org/junit5/docs/current/user-guide/ (Accessed: 5 September 2026).

LIST OF FIGURES

LIST OF TABLES

st20286243 CIS6003 WRIT1.pdf

ORIGINALITY REPORT

7%	5%	2%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Wales Institute, Cardiff Student Paper	1%
2	blogs.kenokivabe.com Internet Source	1%
3	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	1%
4	Submitted to Asia Pacific International College Student Paper	1%
5	Submitted to Kingston University Student Paper	<1%
6	Submitted to West Los Angeles College Student Paper	<1%
7	doczz.net Internet Source	<1%
8	Submitted to University of Portsmouth Student Paper	<1%
9	Submitted to John's Creek High School Student Paper	<1%
10	Submitted to University of Brighton Student Paper	<1%
11	www.theseus.fi Internet Source	<1%
12	Submitted to Universiti Teknologi Petronas Student Paper	<1%
13	flowcraft.app Internet Source	<1%
14	Submitted to Coventry University Student Paper	<1%
15	Submitted to Berlin School of Business and Innovation Student Paper	<1%

16	Submitted to Webster University Student Paper	<1%
17	programmer.group Internet Source	<1%
18	juejin.cn Internet Source	<1%
19	lists.jboss.org Internet Source	<1%
20	stackoverflow.com Internet Source	<1%
21	www.devopsschool.com Internet Source	<1%
22	10-5.jp Internet Source	<1%
23	Submitted to Swinburne University of Technology Student Paper	<1%
24	codewithphp.com Internet Source	<1%
25	Submitted to Yildirim Beyazit Universitesi Student Paper	<1%
26	github.com Internet Source	<1%
27	Submitted to UC, Irvine Student Paper	<1%
28	gitlab.fdmci.hva.nl Internet Source	<1%
29	vdoc.pub Internet Source	<1%
30	www.mchweb.net Internet Source	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		